

STM32F407 UDS Flash Bootloader

Software Architecture Document

Tài liệu	Software Architecture Document (SAD) – kiến trúc, đặc tả, kế hoạch triển khai
Dự án	FBL-F407: flash bootloader theo UDS ISO 14229, transport UART, sẵn sàng chuyển sang CAN
Phần cứng	STM32F407G-DISC1 (STM32F407VG: Cortex-M4F, 1 MB flash, 192 KB SRAM)
Công cụ	STM32CubeIDE, Git, Python 3
Phiên bản	3.1 – thay thế hoàn toàn v1.0 và v2.0
Ngày	24/09/2026
Tác giả	Duong
Trạng thái	Baseline – dùng làm căn cứ triển khai

Tài liệu mô tả đầy đủ bootloader ở mức kiến trúc: mục tiêu, yêu cầu, các quyết định thiết kế và lý do, bố trí bộ nhớ, các module và giao diện giữa chúng, hành vi lúc chạy, giao thức, driver, kiểm chứng và kế hoạch theo tuần. Tài liệu không chứa mã nguồn; mã được viết theo tài liệu này.

Lịch sử phiên bản

Phiên bản	Ngày	Nội dung
1.0	24/09/2026	SRS chi tiết, hai target F407/F411, nhiều tính năng nâng cao
2.0	24/09/2026	Hướng dẫn rút gọn theo bước, chỉ F407
3.0	24/09/2026	Gộp và thay thế: kiến trúc đầy đủ, UDS theo chuẩn, tách transport cho CAN, kế hoạch theo work package
3.1	24/09/2026	Làm rõ quyết định kiến trúc (đã chọn / bị loại); thêm tài liệu UDS; thêm cấu trúc project đầy đủ; thêm WBS chia việc đến từng task

Mục lục

1. Giới thiệu	5
1.1 Mục đích tài liệu	5
1.2 Cách đọc	5
1.3 Tài liệu tham chiếu	5
1.4 Tài liệu UDS (ISO 14229) và cách dùng	6
1.5 Thuật ngữ	6
2. Mục tiêu, phạm vi và ràng buộc	8
2.1 Mục tiêu	8
2.2 Tiêu chí thành công	8
2.3 Phạm vi	8
2.4 Ràng buộc	8
2.5 Giả định	9
3. Tổng quan hệ thống	10
3.1 Ngữ cảnh	10
3.2 Phần cứng	10
3.3 Phân bố chân	10
3.4 Các tình huống sử dụng	11
4. Yêu cầu	12
4.1 Yêu cầu chức năng	12
4.2 Yêu cầu phi chức năng	12
5. Quyết định kiến trúc	13
6. Kiến trúc bộ nhớ	15
6.1 Memory map	15
6.2 Bản ghi META	15
6.3 Yêu cầu với image của app	16
6.4 BOOT FLAG	16
6.5 Ngân sách tài nguyên bootloader	16
7. Kiến trúc phần mềm	17
7.1 Phân lớp	17
7.2 Danh mục module	17

7.3 Quy tắc phụ thuộc	18
7.4 Hợp đồng giao diện PduR	18
7.5 Mô hình thực thi	18
7.6 Chiến lược xử lý lỗi	18
8. Hành vi lúc chạy	20
8.1 Khởi động	20
8.2 Chuyển giao sang application	20
8.3 Vào bootloader từ application	21
8.4 Flashing sequence	21
8.5 State machine lập trình	21
8.6 Phân tích mất nguồn	22
9. Giao tiếp: transport UART và đường sang CAN	23
9.1 Lớp vật lý	23
9.2 Định dạng frame	23
9.3 Nhận frame	23
9.4 Chính sách buffer	23
9.5 Chuyển sang CAN (giai đoạn 2)	24
10. Đặc tả chẩn đoán (UDS)	25
10.1 Quy ước bản tin	25
10.2 Session và timing	25
10.3 Thứ tự kiểm tra NRC	25
10.4 Bảng service	25
10.5 Đặc tả từng service	26
10.6 Data Identifier	26
10.7 Routine Identifier	27
10.8 Security Access	27
11. Drivers	28
12. Application hợp tác	29
13. Tool phía PC	30
13.1 Kiến trúc tool	30
13.2 Tái sử dụng từ STM32_Programmer_V1.py	30
14. Kiểm chứng	31
14.1 Chiến lược	31
14.2 Danh sách test hệ thống	31
14.3 Đo đặc cần ghi lại	32
15. Cấu trúc project	33
15.1 Cây thư mục đầy đủ	33
15.2 Vai trò từng thư mục	34

15.3 File nào được tạo ở work package nào	35
15.4 Chia sẻ common/ trong STM32CubeIDE	35
16. Quy trình phát triển	36
16.1 Git	36
16.2 Quy tắc mã nguồn	36
16.3 Tiêu chí hoàn thành chung cho mỗi work package	36
17. Kế hoạch triển khai và timeline	37
17.1 Timeline	37
17.2 Work packages	37
18. Phân rã công việc (WBS)	40
18.1 Nguyên tắc chia từ lớn đến nhỏ	40
18.2 Tổng hợp	40
18.3 WP0 – Nền tảng (≈ 6 h)	40
18.4 WP1 – Boot core (≈ 10 h)	41
18.5 WP2 – Drivers (≈ 27 h)	41
18.6 WP3 – Transport UART (≈ 10.5 h)	42
18.7 WP4 – Dcm core (≈ 15 h)	42
18.8 WP5 – Lập trình flash (≈ 20 h)	42
18.9 WP6 – Toàn vẹn (≈ 10 h)	43
18.10 WP7 – Security (≈ 5 h)	43
18.11 WP8 – App hợp tác (≈ 10 h)	43
18.12 WP9 – Kiểm chứng và bàn giao (≈ 15 h)	44
18.13 WP10 – CAN (≈ 30 h)	44
19. Rủi ro	45
20. Chuẩn bị phòng vấn	46
21. Phụ lục	47
21.1 Phụ lục A – Bảng NRC	47
21.2 Phụ lục B – Ngoài phạm vi và hướng nâng cấp	47

1. Giới thiệu

1.1 Mục đích tài liệu

Tài liệu là nguồn duy nhất mô tả bootloader cần xây dựng. Người đọc (chính bạn, người review, người phỏng vấn) có thể hiểu hệ thống làm gì, vì sao được thiết kế như vậy, mỗi phần được hiện thực ở module nào, kiểm tra thế nào và làm theo thứ tự nào.

1.2 Cách đọc

Muốn biết	Đọc chương
Dự án nhằm đạt gì, phạm vi đến đâu	1, 2
Hệ thống gồm những gì, chạy trên phần cứng nào	3
Hệ thống phải làm được gì (yêu cầu)	4
Vì sao chọn thiết kế này	5
Bộ nhớ được chia thế nào	6
Phần mềm gồm module nào, giao tiếp ra sao	7
Hệ thống chạy thế nào theo thời gian	8
Giao thức UART và cách chuyển sang CAN	9
Đặc tả UDS từng service	10
Mỗi driver làm gì	11
App và tool PC cần làm gì	12, 13
Kiểm tra thế nào	14
Thư mục project và từng file	15
Quy trình, kế hoạch, timeline	16, 17
Chia việc từ lớn đến nhỏ (WBS)	18
Rủi ro, chuẩn bị phòng vấn	19, 20

1.3 Tài liệu tham chiếu

Mã	Tài liệu	Dùng cho
[ISO14229-1]	ISO 14229-1 – UDS, application layer	Service, NRC, flashing sequence
[ISO14229-2]	ISO 14229-2 – UDS, session layer services	P2, P2*, S3
[ISO15765-2]	ISO 15765-2 – Transport protocol trên CAN (ISO-TP)	Giai đoạn 2
[RM0090]	ST RM0090 – Reference manual STM32F405/407	Thanh ghi mọi peripheral
[PM0214]	ST PM0214 – Cortex-M4 programming manual	NVIC, SysTick, VTOR
[DS]	Datasheet STM32F407xx	Thời gian erase/program flash
[UM1472]	ST UM1472 – STM32F4DISCOVERY user manual	Chân LED, nút
[AN4187]	ST AN4187 – Using the CRC peripheral on STM32	CRC32 phần cứng
[NIEKIRAN]	github.com/niekiran/BootloaderProjectSTM32	Host Python gốc
[DEVLINUX]	DevLinux – BootLoader Mock Project	Tư tưởng phân lớp

1.4 Tài liệu UDS (ISO 14229) và cách dùng

UDS được định nghĩa trong bộ chuẩn ISO 14229 gồm nhiều phần. Dự án chỉ cần phần 1 (application layer) và phần 2 (session layer); phần 3 (UDS on CAN) và ISO 15765-2 dùng ở giai đoạn CAN.

Tài liệu	Loại	Nội dung / dùng để	Nguồn
ISO 14229-1 – Application layer	Chuẩn (trả phí)	Định nghĩa mọi service, sub-function, NRC, định dạng bản tin. Bản 2020 (466 trang) được dùng rộng rãi; bản 2026 đã phát hành thay thế – nếu dùng bản mới, đối chiếu lại các service trong dự án	iso.org/standard/72439.html
ISO 14229-2 – Session layer services	Chuẩn (trả phí)	Timing P2, P2*, S3 và hành vi session	iso.org (bản 2021)
ISO 14229-3 – UDS on CAN	Chuẩn (trả phí)	Yêu cầu riêng khi UDS chạy trên CAN	iso.org
ISO 15765-2 – ISO-TP	Chuẩn (trả phí)	Transport trên CAN: SF, FF, CF, FC, timing	iso.org
Bản xem trước ISO 14229-1:2020	Miễn phí (vài trang đầu)	Phạm vi, mô hình OSI, khái niệm client/server	standards.iteh.ai
CSS Electronics – UDS Explained	Bài hướng dẫn	Cấu trúc bản tin, service chính, ISO-TP, ví dụ thực tế	csselectronics.com/pages/uds-protocol-tutorial-unified-diagnostic-services
Wikipedia – Unified Diagnostic Services	Tổng quan	Bảng SID, NRC, quan hệ với CAN/DoIP/AUTOSAR	en.wikipedia.org/wiki/Unified_Diagnostic_Services
EmbeTronicX – UDS protocol tutorial	Loạt bài hướng dẫn	Giải thích từng service kèm ví dụ byte	embetronicx.com/tutorials/automotive/uds-protocol
driftregion/iso14229	Mã nguồn mở (C)	UDS server/client cho hệ nhúng, cấp phát tĩnh, có ví dụ firmware update – đọc để đối chiếu cách xử lý NRC, không copy vào dự án	github.com/driftregion/iso14229
python-udsoncan	Mã nguồn mở (Python)	Client UDS theo ISO 14229; đọc để hiểu phía tester, có thể dùng để kiểm tra chéo bootloader	github.com/pylessard/python-udsoncan
python-can-isotp	Mã nguồn mở (Python)	ISO-TP phía PC cho giai đoạn CAN	github.com/pylessard/python-can-isotp
Khóa Udemy về automotive bootloader + UDS	Khóa học	P2/P2*/S3, trình tự lập trình, CAPL cơ bản	udemy.com

Thứ tự đọc đề xuất: (1) CSS Electronics và Wikipedia để có bức tranh chung – 1 buổi; (2) EmbeTronicX cho từng service dùng trong dự án (10, 11, 22, 27, 2E, 31, 34, 36, 37, 3E, 28, 85) – trước WP4; (3) mã driftregion/iso14229 phần server để đối chiếu cách kiểm tra NRC – trong WP4-WP5; (4) python-udsoncan để kiểm tra chéo – WP9; (5) nếu có điều kiện, mua ISO 14229-1 để đối chiếu chính xác định dạng từng service.

Nguyên tắc: Mọi định dạng bản tin trong Chương 10 được viết theo cách hiểu phổ biến của ISO 14229-1:2020. Khi có bản chuẩn trong tay, rà lại từng service; ghi mọi chỗ lệch vào nhật ký dự án.

1.5 Thuật ngữ

Thuật ngữ	Ý nghĩa
Bootloader / FBL	Phần mềm chạy đầu tiên sau reset; quyết định chạy app hay cập nhật app
Application (app)	Phần mềm chức năng được bootloader nạp và khởi chạy
Tester / Client	Phía gửi request UDS (tool Python trên PC)
Server / ECU	Phía trả lời (board STM32)

Thuật ngữ	Ý nghĩa
UDS	Unified Diagnostic Services – giao thức chẩn đoán và nạp phần mềm dùng trong ô tô
SID	Service Identifier – byte đầu của request
NRC	Negative Response Code – mã lỗi trong response 7F SID NRC
DID / RID	Data Identifier (đọc/ghi dữ liệu) / Routine Identifier (chạy thủ tục)
SPRMIB	Bit 7 của sub-function: yêu cầu server không gửi positive response
BSC	Block sequence counter trong TransferData
P2 / P2* / S3	Thời gian trả lời thường / sau NRC 0x78 / timeout của session
PDU	Một bản tin UDS hoàn chỉnh (SID + dữ liệu)
META	Vùng flash lưu trạng thái app (hợp lệ hay không, kích thước, CRC)
Boot flag	Vùng RAM app dùng để yêu cầu bootloader ở lại lập trình sau reset
VTOR / MSP	Thanh ghi địa chỉ vector table / con trỏ stack chính của Cortex-M4
Physical / Functional	Request gửi riêng một ECU / gửi đồng thời mọi ECU

2. Mục tiêu, phạm vi và ràng buộc

2.1 Mục tiêu

Mục tiêu chính. Xây dựng một flash bootloader cho STM32F407 cập nhật application qua giao thức UDS theo ISO 14229, đi đúng flashing sequence như ECU ô tô, không bao giờ khởi chạy một application hỏng, và có kiến trúc tách lớp transport để chuyển từ UART sang CAN mà không sửa phần còn lại.

Mục tiêu nghề nghiệp. Dự án là bằng chứng năng lực cho vị trí Flash Bootloader Developer: lập trình C trên thanh ghi, hiểu memory layout và flash, UDS, security access, và cách kiểm chứng một phần mềm quan trọng.

2.2 Tiêu chí thành công

ID	Tiêu chí	Cách đo
SC-1	Nạp app v1 → v2 → v1 hoàn toàn bằng tool Python, không chạm tay vào board	Test HIL
SC-2	Mất nguồn ở bất kỳ bước nào của quá trình nạp: board luôn vào lại bootloader và nạp lại được	Test mất nguồn
SC-3	Mọi NRC trong đặc tả có test tương ứng và pass	Báo cáo test
SC-4	Bootloader ≤ 32 KB flash	Map file
SC-5	Chuyển sang CAN: Dcm và các module Services không có thay đổi mã nguồn	git diff giữa hai tag
SC-6	Người khác đọc README và nạp được app trong 15 phút	Thử với một người

2.3 Phạm vi

Trong phạm vi	Ngoài phạm vi
Bootloader với driver tự viết trên thanh ghi	AUTOSAR stack thật (chỉ mượn tên và vai trò module)
UDS: session, security, download, routine, đọc/ghi DID, NRC, timing	Chữ ký số, mã hóa image
Transport UART tự định nghĩa, tương đương ISO-TP về vai trò	Cập nhật chính bootloader
Kiểm tra app hợp lệ, an toàn khi mất nguồn (single slot)	Dual-bank A/B, staging, rollback
Demo app hợp tác (vào bootloader bằng lệnh UDS)	Nén dữ liệu, nhiều logical block
Tool Python + bộ test tự động trên board	GUI, CANoe/CAPL
Giai đoạn 2: CAN + ISO-TP	CAN-FD, Ethernet

Lý do các mục ngoài phạm vi: mỗi mục tốn nhiều tuần nhưng không phải trọng tâm đánh giá. Tài liệu vẫn mô tả chúng ở mức khái niệm (Phụ lục B) để có thể giải thích khi được hỏi.

2.4 Ràng buộc

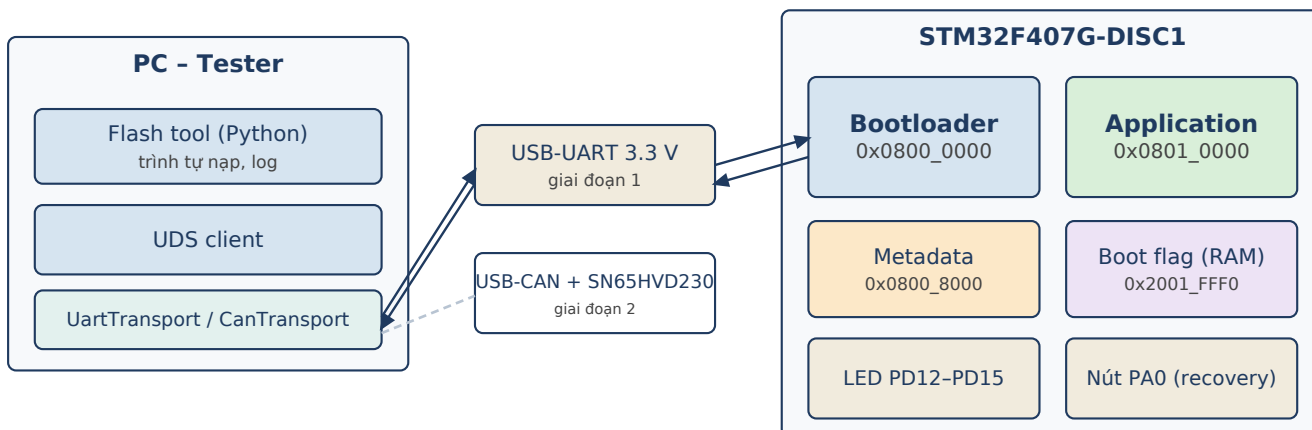
- Một board STM32F407G-DISC1; không có debugger ngoài ST-LINK onboard.
- Bootloader không dùng HAL/LL; chỉ dùng CMSIS (định nghĩa thanh ghi). App được phép dùng HAL.
- Không RTOS, không cấp phát động, không đệ quy trong bootloader.
- Mọi thứ phải đơn giản đủ để một người làm ngoài giờ trong khoảng 12 tuần (chưa tính CAN).

2.5 Giả định

- Nguồn cấp ổn định trong lúc chạy bình thường; mất nguồn chỉ xét như sự cố.
- Tester tuân thủ hỏi-đáp: gửi một request, chờ response rồi mới gửi tiếp.
- Một tester tại một thời điểm.

3. Tổng quan hệ thống

3.1 Ngữ cảnh



Hình 3.1 – Ngữ cảnh hệ thống

PC chạy tool Python đóng vai tester. Tool nói UDS với board qua một USB-UART (giai đoạn 1) hoặc USB-CAN (giai đoạn 2). Trên board, bootloader nằm ở đầu flash, application ở phía sau, giữa hai phần là vùng META lưu trạng thái application. Boot flag trong RAM là kênh để application nhờ bootloader ở lại sau reset.

3.2 Phần cứng

Thiết bị	Bắt buộc	Vai trò
STM32F407G-DISC1	Có	Target; ST-LINK onboard để nạp bootloader lần đầu và debug
USB-UART 3.3 V (CP2102/CH340/FT232)	Có	Kênh UDS giữa PC và board
Dây jumper, cáp mini-USB	Có	Nối dây, cấp nguồn
Logic analyzer 8 kênh 24 MHz	Nên có	Quan sát frame, đo thời gian, lấy hình cho tài liệu
SN65HVD230 + USB-CAN (CANable) + 120 Ω	Giai đoạn 2	CAN bus

3.3 Phân bổ chân

Chức năng	Chân	Cấu hình
UDS TX	PA2	USART2_TX, AF7, push-pull
UDS RX	PA3	USART2_RX, AF7, pull-up
Nút recovery	PA0	Input, mức cao khi nhấn
LED xanh lá	PD12	Nháy 1 Hz: bootloader, Default session
LED cam	PD13	Sáng: Programming session
LED đỏ	PD14	Sáng: lỗi gần nhất (flash, CRC)
LED xanh dương	PD15	Đào mỗi khi nhận một request hợp lệ
CAN RX/TX (giai đoạn 2)	PB8 / PB9	CAN1, AF9 – kiểm tra xung đột với board trước khi dùng

Kiểm tra: Đối chiếu từng chân với [UM1472] trước khi đấu dây để tránh chân đã nối với linh kiện onboard.

3.4 Các tình huống sử dụng

ID	Tình huống	Mô tả ngắn
UC-1	Khởi động bình thường	Reset → bootloader thấy app hợp lệ → chạy app
UC-2	Cập nhật từ app	Tester gửi lệnh cho app → app đặt boot flag, reset → bootloader nhận nạp → reset → app mới
UC-3	Cập nhật khi chưa có app	Board mới hoặc app đã bị xóa → bootloader ở lại, tester nạp trực tiếp
UC-4	Recovery bằng nút	App chạy nhưng không nhận lệnh → giữ PA0 khi reset → ở lại bootloader
UC-5	Nạp thất bại / mất nguồn	Lần khởi động sau bootloader phát hiện app không hợp lệ → ở lại chờ nạp lại
UC-6	Truy vấn thông tin	Tester đọc version bootloader/app, session, serial, fingerprint

4. Yêu cầu

Mỗi yêu cầu có ID để truy vết sang test (Chương 14). Ưu tiên: M = bắt buộc cho phiên bản 1.0, S = nên có, C = có thể làm nếu còn thời gian.

4.1 Yêu cầu chức năng

ID	Yêu cầu	Ưu tiên
FR-01	Sau reset, bootloader quyết định chạy app hay ở lại theo thứ tự: boot flag → nút → app hợp lệ	M
FR-02	Chỉ chạy app khi META hợp lệ, CRC32 của app khớp, MSP và reset vector hợp lý	M
FR-03	Chuyển giao sang app với trạng thái MCU gần như sau reset (ngắt tắt, peripheral về mặc định, VTOR của app)	M
FR-04	Nhận request UDS qua UART theo frame định nghĩa ở Chương 9, trả response cùng định dạng	M
FR-05	Hỗ trợ các service ở Chương 10 với đúng định dạng request/response và NRC	M
FR-06	Chỉ cho xóa/ghi flash sau khi vào Programming session và mở khóa Security Access	M
FR-07	Chỉ cho xóa/ghi trong vùng APP; mọi địa chỉ khác bị từ chối	M
FR-08	Bắt buộc đúng thứ tự lập trình; sai thứ tự trả NRC 0x24	M
FR-09	Đánh dấu app hợp lệ chỉ sau khi CRC toàn image khớp và kiểm tra phụ thuộc đạt	M
FR-10	App vào bootloader bằng request 10 02; bootloader trả lời thay app	M
FR-11	Tự thoát Programming session nếu không có request trong S3	M
FR-12	Cung cấp thông tin: version bootloader/app, session, serial, fingerprint, trạng thái app	M
FR-13	Ghi fingerprint trước khi xóa; lưu cùng META	S
FR-14	Giới hạn số lần nhập key sai, sau đó áp thời gian chờ	S
FR-15	Đếm số lần lập trình, lỗi frame; đọc được qua DID	C

4.2 Yêu cầu phi chức năng

ID	Nhóm	Yêu cầu
NFR-01	Kích thước	Bootloader $\leq$ 32 KB flash, $\leq$ 16 KB RAM
NFR-02	Thời gian	Reset đến lúc nhảy sang app $\leq$ 500 ms với app 448 KB (đo và ghi lại)
NFR-03	Timing UDS	Bắt đầu gửi response trong P2 = 50 ms, hoặc gửi 0x78 và trả lời cuối trong P2* = 5000 ms
NFR-04	Thông lượng	$\geq$ 9 KB/s ở 115200 baud; baud cấu hình được đến 921600
NFR-05	Độ tin cậy	Không tồn tại chuỗi sự kiện khiến board chạy app hỏng hoặc mất khả năng vào bootloader
NFR-06	Khả chuyển	Thay UART bằng CAN chỉ thêm driver/transport và đổi cấu hình PduR
NFR-07	Bảo trì	Mỗi module một cặp .c/.h; không có biến toàn cục dùng chung giữa module
NFR-08	Watchdog	Mọi vòng chờ có timeout; IWDG reset hệ thống nếu treo
NFR-09	Kiểm chứng	Mọi yêu cầu M có ít nhất một test tự động

5. Quyết định kiến trúc

Mỗi quyết định gồm: phương án **đã chọn** (được thực hiện trong dự án), các phương án **bị loại** (đã cân nhắc nhưng không làm) và lý do. Đây là phần nên đọc kỹ trước phỏng vấn.

AD-01 - Single slot + valid flag	
Đã chọn	Single slot + valid flag
Phương án bị loại	Staging (vùng tạm) + copy; dual slot A/B chạy tại chỗ
Lý do / trade-off	Đúng cách FBL ô tô làm: nạp lỗi thì ECU ở lại bootloader chờ nạp lại. Đơn giản, dùng hết flash. Không rollback được về app cũ. Memory map vẫn chứa 512 KB để nâng lên staging sau này.
AD-02 - UDS theo ISO 14229 cho tầng ứng dụng	
Đã chọn	UDS theo ISO 14229 cho tầng ứng dụng
Phương án bị loại	Bộ lệnh tự định nghĩa: kiểu khóa niekiran (0x51 GET_VER, 0x57 MEM_WRITE, ACK/NACK) hoặc giao thức bootloader ROM của ST (AN3155)
Lý do / trade-off	Khớp JD; tester và NRC có ý nghĩa chuẩn; chuyển sang CAN tự nhiên.
AD-03 - Transport UART tự định nghĩa, có byte địa chỉ	
Đã chọn	Transport UART tự định nghĩa, có byte địa chỉ
Phương án bị loại	Luồng byte không có frame; frame dùng byte-stuffing (COBS/SLIP)
Lý do / trade-off	Frame giúp biết ranh giới bản tin và phát hiện lỗi; byte địa chỉ giả lập physical/functional như CAN.
AD-04 - Interface PduR cố định giữa transport và Dcm	
Đã chọn	Interface PduR cố định giữa transport và Dcm
Phương án bị loại	Dcm đọc trực tiếp từ UART
Lý do / trade-off	Dcm không biết transport là gì → thêm CAN không sửa Dcm.
AD-05 - CRC32 bằng peripheral CRC của STM32, mỗi byte nạp như một word	
Đã chọn	CRC32 bằng peripheral CRC của STM32, mỗi byte nạp như một word
Phương án bị loại	CRC-16 phần mềm; CRC-32 chuẩn (zlib); CRC phần cứng nạp theo word
Lý do / trade-off	Dùng phần cứng có sẵn, một cách tính cho cả frame và image, khớp hàm get_crc() của host niekiran. Đổi lại chậm hơn nạp theo word khoảng 4 lần.
AD-06 - Clock HSI 16 MHz, không bật PLL	
Đã chọn	Clock HSI 16 MHz, không bật PLL
Phương án bị loại	Bật PLL 168 MHz
Lý do / trade-off	Không phải trả clock về trước khi nhảy; app tự cấu hình clock. Đổi lại CRC lúc boot chậm hơn. Nếu đo thấy vượt NFR-02, chuyển sang PLL và thêm bước khôi phục HSI trước khi nhảy.
AD-07 - Ring buffer byte trong ngắt RX, không queue frame	
Đã chọn	Ring buffer byte trong ngắt RX, không queue frame
Phương án bị loại	Polling; DMA + ngắt IDLE; queue 4 frame
Lý do / trade-off	USART chỉ giữ 1 byte, cần buffer để không mất byte. UDS hỏi-đáp nên chỉ có tối đa một frame chờ xử lý.
AD-08 - Vòng lặp cooperative, không RTOS	
Đã chọn	Vòng lặp cooperative, không RTOS
Phương án bị loại	FreeRTOS
Lý do / trade-off	Bootloader nhỏ, luồng tuần tự; không cần lập lịch; dễ kiểm chứng.

AD-09 – Driver tự viết trên thanh ghi, đặt tên kiểu ST	
Đã chọn	Driver tự viết trên thanh ghi, đặt tên kiểu ST
Phương án bị loại	HAL; LL
Lý do / trade-off	Thể hiện hiệu phần cứng; mã nhỏ; tên hàm quen thuộc với người đọc code ST.

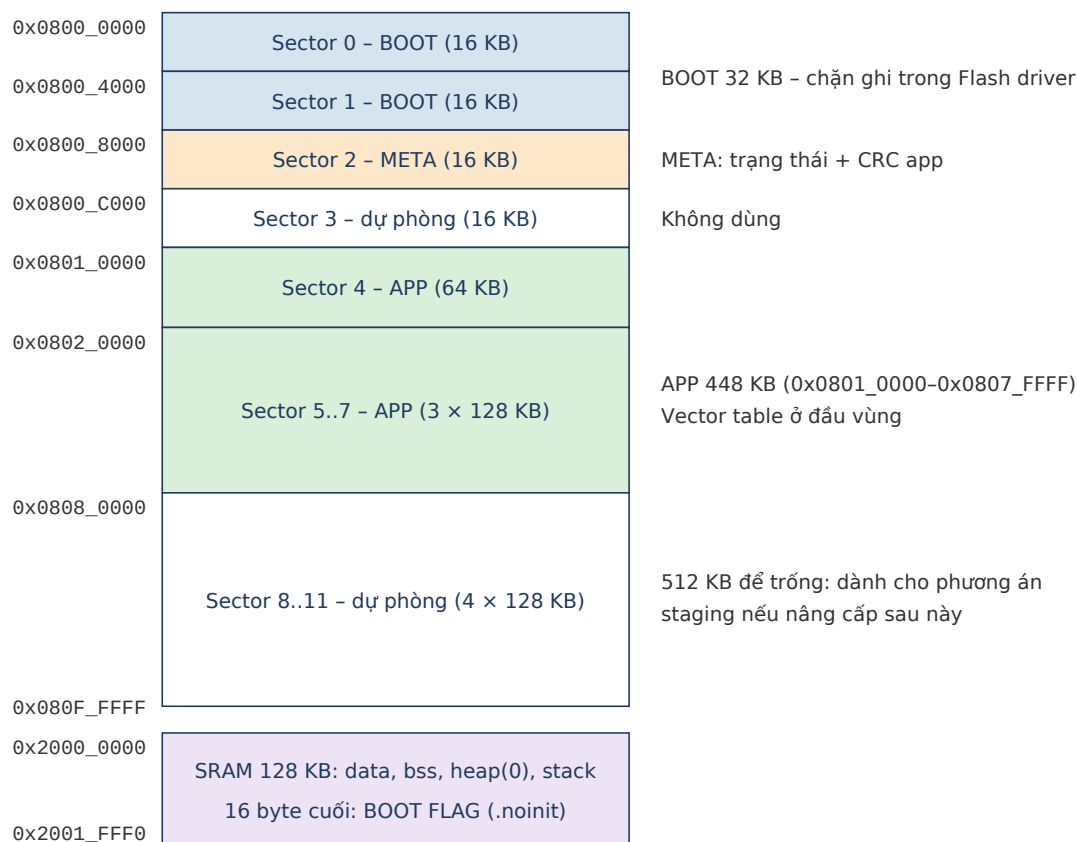
AD-10 – Tên module mượn từ AUTOSAR (Dcm, PduR, CanTp...)	
Đã chọn	Tên module mượn từ AUTOSAR (Dcm, PduR, CanTp...)
Phương án bị loại	Tên tự đặt
Lý do / trade-off	Người phỏng vấn automotive hiểu ngay vai trò từng module.

AD-11 – META một sector, xóa đầu tiên, ghi sau cùng	
Đã chọn	META một sector, xóa đầu tiên, ghi sau cùng
Phương án bị loại	EEPROM emulation hai sector A/B
Lý do / trade-off	Đủ để an toàn khi mất nguồn trong phạm vi single slot; bộ đếm có thể mất khi mất nguồn – chấp nhận.

AD-12 – Security Access thuật toán demo, seed từ RNG phần cứng	
Đã chọn	Security Access thuật toán demo, seed từ RNG phần cứng
Phương án bị loại	AES-CMAC; không có security
Lý do / trade-off	Thể hiện đủ cơ chế seed/key và giới hạn số lần thử; thuật toán thay được sau.

6. Kiến trúc bộ nhớ

6.1 Memory map



Hình 6.1 – Memory map STM32F407VG

Vùng	Sector	Địa chỉ	Kích thước	Nội dung
BOOT	0-1	0x0800_0000	32 KB	Bootloader
META	2	0x0800_8000	16 KB	Bản ghi trạng thái app
Dự phòng	3	0x0800_C000	16 KB	Để trống
APP	4-7	0x0801_0000	448 KB	Application
Dự phòng / staging	8-11	0x0808_0000	512 KB	Để trống cho nâng cấp
BOOT FLAG	RAM	0x2001_FFF0	16 B	Yêu cầu vào bootloader từ app

Lý do: sector 0-3 là bốn sector nhỏ nhất (16 KB) nên dùng cho bootloader và META; xóa META không đựng code. APP bắt đầu ở đầu sector 4 (0x0801_0000), căn lề thỏa yêu cầu của VTOR. 448 KB là quá đủ cho demo; phần còn lại để trống giữ chỗ cho phương án staging.

6.2 Bản ghi META

Trường	Kích thước	Ý nghĩa
magic	4 B	Giá trị cố định đánh dấu bản ghi hợp lệ (khác 0xFFFFFFFF của flash trống)
app_size	4 B	Số byte của app (bội số của 4)
app_crc	4 B	CRC32 STM32 trên toàn bộ app
app_version	4 B	Phiên bản app do tester cung cấp (major.minor.patch)
fingerprint	12 B	Ngày lập trình (3 B BCD) + ID tester (6 B) + đệm
prog_count	4 B	Số lần lập trình thành công
meta_crc	4 B	CRC32 trên các trường phía trước, phát hiện bản ghi ghi dở

Thao tác	Khi nào	Hệ quả
Đọc	Mỗi lần khởi động; khi đọc DID	Bản ghi hợp lệ khi magic và meta_crc đúng
Xóa (vô hiệu hóa)	Đầu tiên trong 31 FF00, trước khi xóa APP	Từ lúc này app được coi là không hợp lệ
Ghi	Chỉ trong 31 FF01 khi mọi kiểm tra đạt	App trở thành hợp lệ

6.3 Yêu cầu với image của app

- Link tại 0x0801_0000, kích thước tối đa 448 KB; vector table ở đầu image.
- App đặt VTOR về địa chỉ của mình (hoặc không ghi đè giá trị bootloader đã đặt).
- Kích thước file .bin được tool đệm 0xFF cho đủ bội số của 4 trước khi gửi.
- RAM của app giảm 16 byte cuối để không đè lên BOOT FLAG.

6.4 BOOT FLAG

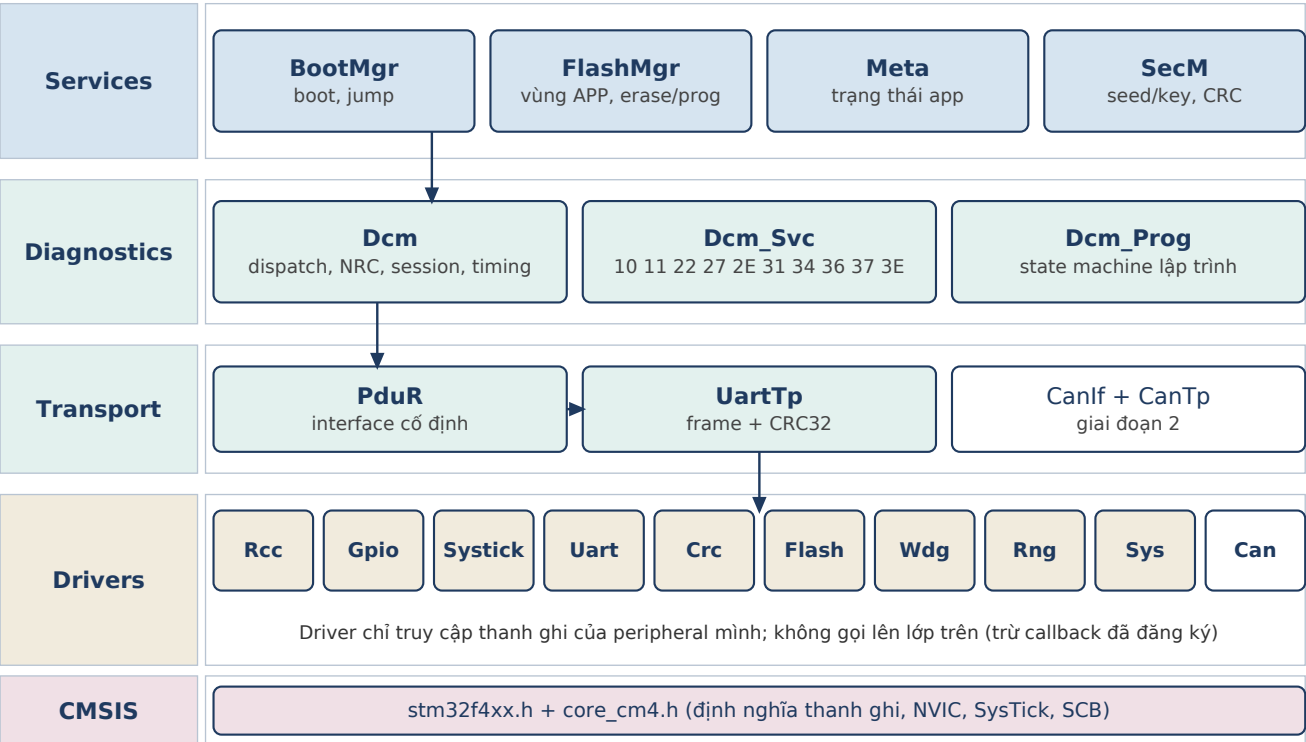
16 byte cuối SRAM, không bị startup khởi tạo, giữ nguyên qua reset mềm. Gồm một giá trị magic và bản đảo bit của nó (giảm khả năng rác RAM lúc bật nguồn trùng magic). Bootloader xóa flag ngay khi đọc được.

6.5 Ngân sách tài nguyên bootloader

Thành phần	Flash ước tính	RAM ước tính
Drivers	4-6 KB	≈ 0.6 KB (ring buffer 512 B)
Transport + PduR	1-2 KB	≈ 1.1 KB (frame buffer)
Dcm + service	4-6 KB	≈ 1.1 KB (response buffer)
Services (BootMgr, FlashMgr, Meta, SecM)	2-3 KB	< 0.2 KB
Startup, vector table	≈ 0.5 KB	Stack 2 KB
Tổng	≈ 12-18 KB (giới hạn 32 KB)	≈ 5 KB

7. Kiến trúc phần mềm

7.1 Phân lớp



Hình 7.1 – Các lớp và module của bootloader

- **Drivers:** làm việc trực tiếp với thanh ghi; không biết UDS là gì.
- **Transport:** biến luồng byte thành PDU hoàn chỉnh và ngược lại; phát hiện lỗi truyền.
- **Diagnostics:** hiểu UDS: session, security state, service, NRC, timing, thứ tự lập trình.
- **Services:** chức năng nghiệp vụ của bootloader: boot, quản lý vùng flash, trạng thái app, security.

7.2 Danh mục module

Module	Lớp	Trách nhiệm	Sử dụng
BootMgr	Services	Quyết định boot, kiểm tra app, deinit và nhảy, xử lý boot flag, reset có kiểm soát	Meta, Sys, Gpio
FlashMgr	Services	Kiểm tra địa chỉ trong vùng APP, erase theo sector, ghi và verify, theo dõi sector đã xóa	Flash, Wdg
Meta	Services	Đọc, vô hiệu hóa, ghi bản ghi META; tính CRC image	Flash, Crc
SecM	Services	Sinh seed, kiểm tra key, đếm số lần sai, thời gian chờ	Rng, Systick
Dcm	Diagnostics	Nhận PDU, tra bảng service, kiểm tra NRC theo thứ tự, quản lý session, P2/P2*/S3, gửi 0x78	PduR, Systick
Dcm_Svc	Diagnostics	Xử lý từng service: phân tích request, gọi Services, tạo response	Services
Dcm_Prog	Diagnostics	State machine lập trình (Hình 8.2)	-
PduR	Transport	Định tuyến PDU giữa transport đang dùng và Dcm	UartTp / CanTp
UartTp	Transport	Frame SOF/ADDR/LEN/PAYLOAD/CRC32, timeout giữa byte, gửi frame	Uart, Crc, Systick
CanIf, CanTp	Transport	Giai đoạn 2: ISO-TP trên CAN	Can

Module	Lớp	Trách nhiệm	Sử dụng
Rcc, Gpio, SysTick, Uart, Crc, Flash, Wdg, Rng, Sys	Drivers	Xem Chương 11	CMSIS

7.3 Quy tắc phụ thuộc

- Chỉ gọi xuống lớp dưới hoặc ngang lớp theo bảng trên; lớp dưới báo lên bằng callback đã đăng ký cố định (ví dụ UartTp gọi PduR khi có PDU).
- Trạng thái của module nằm trong chính module đó; module khác chỉ truy cập qua hàm.
- Chỉ Drivers được include header thanh ghi của chip. Transport, Diagnostics, Services không phụ thuộc phần cứng nên kiểm thử được trên PC.
- Mọi hằng số cấu hình (địa chỉ, timing, DID, baud) nằm trong một file cấu hình duy nhất.

7.4 Hợp đồng giao diện PduR

Đây là giao diện quan trọng nhất của kiến trúc: giữ nguyên thì chuyển sang CAN không phải sửa Dcm.

Hàm	Hướng	Ý nghĩa	Khi nào gọi
PduR_RxIndication(pdu, len, addr)	Transport → Dcm	Đã nhận đủ một request UDS; addr = physical / functional	Frame hợp lệ (UART) hoặc ISO-TP hoàn tất (CAN)
PduR_Transmit(pdu, len)	Dcm → Transport	Gửi một response UDS	Dcm có response
PduR_TxConfirmation(result)	Transport → Dcm	Response đã rời khỏi thiết bị	Byte cuối gửi xong (UART) / ISO-TP hoàn tất (CAN)
Tp_MainFunction()	Main → Transport	Xử lý byte đến, timeout	Mỗi vòng lặp chính
Dcm_MainFunction()	Main → Dcm	Timing P2, P2*, S3; gửi 0x78	Mỗi vòng lặp chính

Kích thước PDU tối đa là 1026 byte (SID + BSC + 1024 byte dữ liệu), dùng chung cho UART và CAN nên maxNumberOfBlockLength trong 0x34 không đổi khi đổi transport.

7.5 Mô hình thực thi

Ngữ cảnh	Nội dung	Quy tắc
Ngắt USART2	Đọc byte, đẩy vào ring buffer, xóa overrun	Ngắn nhất có thể; không gọi hàm lớp trên
Ngắt SysTick 1 ms	Tăng bộ đếm tick	Chỉ tăng biến
Vòng lặp chính	Wdg refresh → Tp_MainFunction → Dcm_MainFunction → cập nhật LED	Không chặn lâu, trừ thao tác erase (có 0x78 trước)

Dữ liệu dùng chung giữa ngắt và vòng lặp chính chỉ có ring buffer (con trỏ ghi do ngắt sở hữu, con trỏ đọc do main sở hữu) và bộ đếm tick; cả hai khai báo volatile, không cần tắt ngắt.

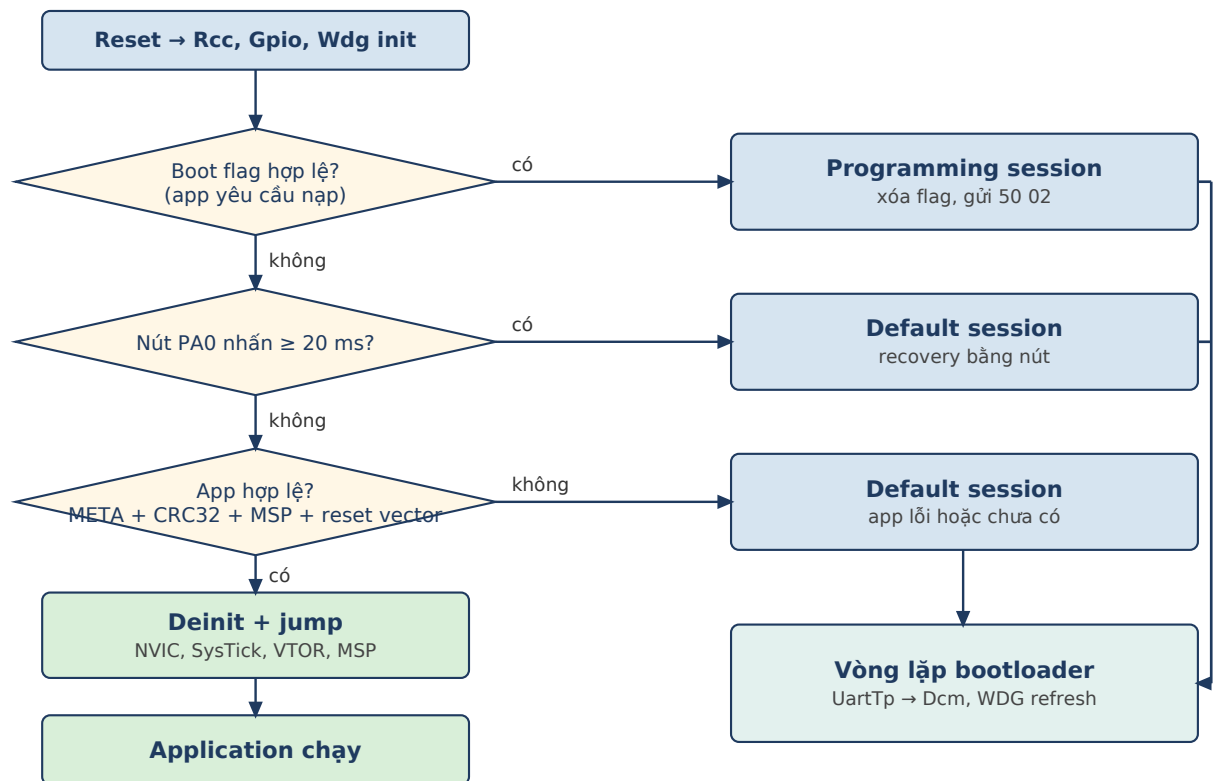
7.6 Chiến lược xử lý lỗi

Loại lỗi	Phát hiện	Xử lý
Lỗi truyền (CRC frame sai, mất byte)	UartTp	Bỏ frame, không trả lời; tester gửi lại
Request sai (độ dài, tham số, thứ tự)	Dcm	NRC tương ứng
Lỗi flash (erase/program/verify)	Flash, FlashMgr	NRC 0x72; LED đỏ; app vẫn không hợp lệ
App không hợp lệ lúc boot	BootMgr, Meta	Ở lại bootloader; ghi lý do vào DID trạng thái
Treo phần mềm	IWDG	Reset; bootloader chạy lại

Loại lỗi	Phát hiện	Xử lý
Tham số API sai (con trỏ NULL, ngoài phạm vi)	Mọi module	Trả mã lỗi, không thực hiện

8. Hành vi lúc chạy

8.1 Khởi động



Hình 8.1 – Quyết định khởi động

Điều kiện app hợp lệ	Lý do
META có magic và meta_crc đúng	Lần lập trình trước đã hoàn tất
CRC32 tính lại trên [0x0801_0000, +app_size) bằng app_crc	Phát hiện image hỏng
Word 0 của app (MSP) nằm trong SRAM 0x2000_0000–0x2002_0000	App link sai RAM
Word 1 (reset vector) nằm trong vùng APP	App link sai flash

8.2 Chuyển giao sang application

1. Tắt ngắt toàn cục.
2. Dừng SysTick và xóa bộ đếm.
3. Tắt mọi ngắt trong NVIC và xóa mọi ngắt đang chờ.
4. Đưa các peripheral đã dùng (USART2, GPIOA, GPIOD, CRC, RNG) về trạng thái reset qua thanh ghi reset của RCC.
5. Đặt VTOR = 0x0801_0000.
6. Nạp MSP bằng word 0 của vector table app.
7. Bật lại ngắt toàn cục (trạng thái giống sau reset; NVIC đã tắt nên không có ngắt nào xảy ra).
8. Nhảy tới reset handler của app (word 1).

Lưu ý: IWDG đã bật thì không tắt được. App phải refresh watchdog ngay từ đầu. Đây là ràng buộc giao diện với app (Chương 12).

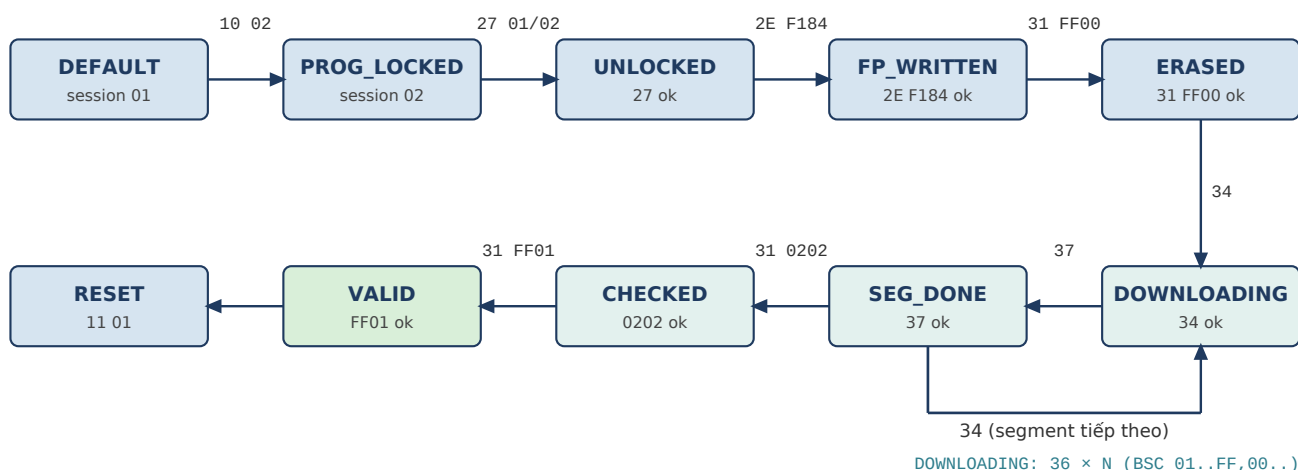
8.3 Vào bootloader từ application

Bước	Tester	Application	Bootloader
1	Gửi 10 02	Nhận, ghi BOOT FLAG, chờ UART gửi xong, reset	-
2	Chờ tối đa P2* (5 s)	-	Thấy BOOT FLAG, xóa flag, vào Programming session
3	Nhận 50 02 ...	-	Gửi 50 02 kèm tham số timing thay cho app

8.4 Flashing sequence

Pha	#	Request	Mục đích
Pre-programming (app)	1	10 03	Vào Extended session
	2	85 02 (functional)	Tắt ghi lỗi DTC
	3	28 03 01 (functional)	Tắt giao tiếp thường
	4	10 02	Yêu cầu vào bootloader
Programming (bootloader)	5	(nhận) 50 02	Bootloader đã sẵn sàng
	6	27 01 → 27 02	Mở khóa
	7	2E F1 84	Ghi fingerprint
	8	31 01 FF 00	Vô hiệu hóa META, xóa vùng APP
	9	34 → 36 × N → 37	Tải image
	10	31 01 02 02	Kiểm tra CRC32 image
	11	31 01 FF 01	Kiểm tra phụ thuộc, ghi META
	12	11 01	Reset
Post-programming (app mới)	13	28 00 01, 85 01 (functional)	Bật lại giao tiếp và DTC
	14	10 01 (functional)	Về Default session

8.5 State machine lập trình



Mọi trạng thái: S3 timeout, 10 01 hoặc lỗi nghiêm trọng -> DEFAULT (khóa security, hủy download).

Service gửi sai trạng thái -> NRC 0x24 requestSequenceError.

Hình 8.2 – Trạng thái lập trình trong Dcm_Prog

Trạng thái	Service được phép	Sự kiện chuyển
DEFAULT	10, 11, 22, 3E	10 02 → PROG_LOCKED
PROG_LOCKED	+ 27, 28, 85	27 02 đúng → UNLOCKED
UNLOCKED	+ 2E	2E F184 → FP_WRITTEN

Trạng thái	Service được phép	Sự kiện chuyển
FP_WRITTEN	+ 31 FF00	31 FF00 đạt → ERASED
ERASED	+ 34	34 đạt → DOWNLOADING
DOWNLOADING	36, 37	37 đạt → SEG_DONE
SEG_DONE	34 (segment khác), 31 0202	31 0202 đúng → CHECKED
CHECKED	31 FF01	FF01 đạt → VALID
VALID	11	11 01 → reset
Mọi trạng thái	-	S3 hết hạn, 10 01 → DEFAULT (khóa lại, hủy download)

8.6 Phân tích mất nguồn

Mất nguồn khi	META sau khi bật lại	Kết quả
Trước 31 FF00	Hợp lệ, app cũ nguyên vẹn	Chạy app cũ
Đang xóa META hoặc APP	Không hợp lệ	Ở lại bootloader, nạp lại được
Đang 36	Không hợp lệ	Ở lại bootloader
Sau 37, trước 31 FF01	Không hợp lệ	Ở lại bootloader
Đang ghi META	meta_crc sai → không hợp lệ	Ở lại bootloader
Sau 31 FF01	Hợp lệ	Chạy app mới

Không có hàng nào dẫn tới chạy app hỏng hoặc không vào được bootloader, vì vùng BOOT không bao giờ bị ghi.

9. Giao tiếp: transport UART và đường sang CAN

9.1 Lớp vật lý

USART2, 115200 baud, 8 bit dữ liệu, không parity, 1 stop bit, không flow control. Baud là một hằng cấu hình (115200 / 460800 / 921600). Nhận bằng ngắt vào ring buffer 512 byte; gửi tuần tự và báo xong khi cờ truyền hoàn tất (TC) bật sau byte cuối.

9.2 Định dạng frame

Trường	Byte	Giá trị	Ghi chú
SOF	1	0xA5	Đánh dấu bắt đầu frame
ADDR	1	0x01 / 0x02	Physical / functional – tương ứng CAN ID 0x7E0 / 0x7DF ở giai đoạn 2
LEN	2	1 ... 1026	Độ dài PAYLOAD, big-endian
PAYLOAD	LEN	PDU UDS	SID + dữ liệu
CRC32	4	-	CRC32 STM32 trên SOF...PAYLOAD, byte thấp gửi trước

Frame response có cùng định dạng với ADDR = 0x01. Giá trị kiểm tra của thuật toán CRC: chuỗi ASCII "123456789" cho 0x1556F485 – dùng để xác nhận MCU và PC tính giống nhau.

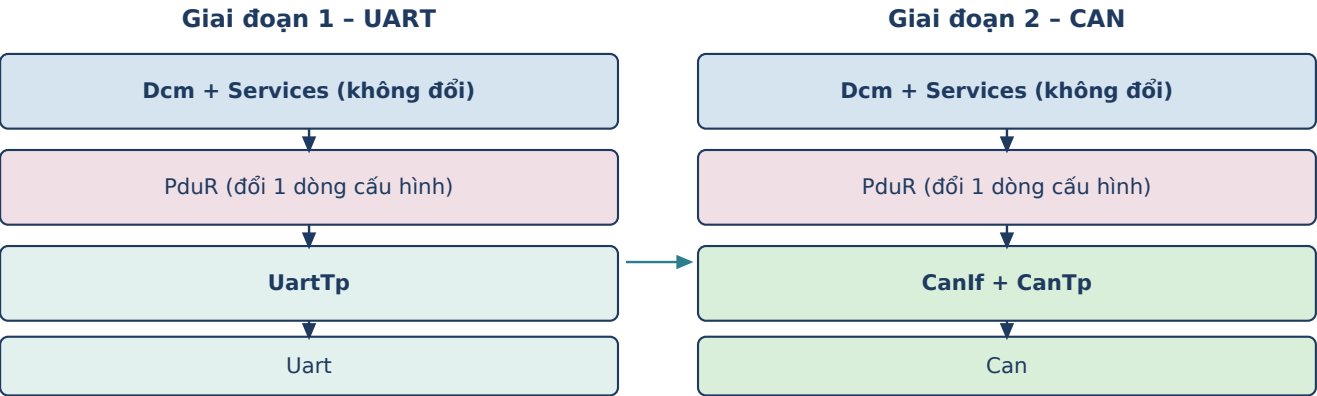
9.3 Nhận frame

Trạng thái	Byte nhận	Hành động
CHỜ SOF	0xA5	Bắt đầu frame mới
CHỜ SOF	khác	Bỏ, đếm lỗi đồng bộ
ADDR	0x01 hoặc 0x02	Lưu kiểu địa chỉ
LEN (2 byte)	-	LEN = 0 hoặc > 1026 → bỏ frame
DATA	LEN byte	Chép vào frame buffer
CRC (4 byte)	-	Đúng → PduR_RxIndication; sai → bỏ, đếm lỗi CRC
Bất kỳ (≠ CHỜ SOF)	không có byte trong 50 ms	Bỏ frame dở, về CHỜ SOF

9.4 Chính sách buffer

- Ring buffer byte 512 byte trong ngắt: bắt buộc, vì USART chỉ giữ 1 byte và mỗi byte đến sau ~87 μ s.
- Một frame buffer: đủ, vì tester chỉ gửi request tiếp khi đã có response.
- Request đến trong lúc Dcm đang xử lý request khác → NRC 0x21 busyRepeatRequest.
- Trong lúc erase flash CPU dừng cả ngắt nên byte đến bị mất. Không ảnh hưởng vì tester đang chờ response.

9.5 Chuyển sang CAN (giai đoạn 2)



Hình 9.1 – Chỉ lớp dưới PduR thay đổi

Hạng mục	Giai đoạn 1	Giai đoạn 2
Driver	Uart	Can (bxCAN, 500 kbit/s, bộ lọc 0x7E0 và 0x7DF)
Transport	UartTp	CanIf + CanTp (single/first/consecutive/flow control frame, STmin, block size, timeout)
Địa chỉ	Byte ADDR	CAN ID: 0x7E0 physical, 0x7DF functional, response 0x7E8
PduR	Trở tới UartTp	Trở tới CanTp (đổi cấu hình)
Dcm, Services	-	Không đổi
Tool PC	UartTransport (pyserial)	CanTransport (python-can + can-isotp)
Bộ test	-	Chạy lại nguyên vẹn

10. Đặc tả chẩn đoán (UDS)

10.1 Quy ước bản tin

- Request: SID + tham số. Positive response: (SID + 0x40) + dữ liệu. Negative response: 7F + SID + NRC.
- Sub-function có bit 7 = SPRMIB. Khi bit này bật và kết quả là positive, không gửi response.
- Request functional (ADDR 0x02): chỉ nhận 10, 11, 3E, 28, 85; không gửi NRC 11, 12, 31, 7E, 7F.

10.2 Session và timing

Session	Mã	Nằm ở	Mục đích
Default	01	Bootloader và app	Mặc định sau reset
Programming	02	Bootloader	Lập trình flash
Extended	03	App	Pre/post-programming (28, 85)

Tham số	Giá trị	Ý nghĩa
P2server	50 ms	Thời gian tối đa đến lúc bắt đầu gửi response
P2*server	5000 ms	Sau NRC 0x78, response tiếp theo phải đến trong khoảng này
S3server	5000 ms	Không có request trong khoảng này ở Programming → về Default
Chu kỳ 0x78	≤ 2000 ms	Nhắc lại 0x78 nếu xử lý còn kéo dài

Response của 10 01 / 10 02 mang P2 (2 byte, đơn vị 1 ms) và P2* (2 byte, đơn vị 10 ms): 50 02 00 32 01 F4. Mọi lần đổi session đều khóa security và hủy download dở.

10.3 Thứ tự kiểm tra NRC

- SID không hỗ trợ → 0x11.
- SID không được phép trong session hiện tại → 0x7F.
- Độ dài nhỏ hơn tối thiểu → 0x13.
- Sub-function không hỗ trợ → 0x12; không được phép trong session → 0x7E.
- Cần mở khóa mà đang khóa → 0x33.
- Độ dài/định dạng chính xác sai → 0x13.
- Sai thứ tự lập trình → 0x24.
- Điều kiện chưa thỏa → 0x22.
- Tham số ngoài phạm vi (địa chỉ, DID, RID, định dạng) → 0x31.
- Lỗi riêng của service (0x35, 0x36, 0x37, 0x70, 0x71, 0x72, 0x73).

10.4 Bảng service

SID	Service	Sub / ID	Default	Prog	Unlock	Functional
10	DiagnosticSessionControl	01, 02	✓	✓	-	✓
11	ECUReset	01	✓	✓	-	✓
22	ReadDataByIdentifier	DID ở 10.6	✓	✓	-	-
27	SecurityAccess	01, 02	-	✓	-	-
28	CommunicationControl	00, 01, 03	-	✓	-	✓
2E	WriteDataByIdentifier	F184	-	✓	✓	-
31	RoutineControl	01 + FF00 / 0202 / FF01	-	✓	✓	-
34	RequestDownload	-	-	✓	✓	-
36	TransferData	-	-	✓	✓	-

SID	Service	Sub / ID	Default	Prog	Unlock	Functional
37	RequestTransferExit	-	-	✓	✓	-
3E	TesterPresent	00	✓	✓	-	✓
85	ControlDTCSetting	01, 02	-	✓	-	✓

10.5 Đặc tả từng service

Service	Request → Positive response	Hành vi và NRC riêng
10	10 xx → 50 xx 00 32 01 F4	01: về Default, khóa, hủy download. 02: vào Programming. Khác → 12.
11	11 01 → 51 01	Reset sau khi response gửi xong. Khác 01 → 12.
3E	3E 00 → 7E 00	Làm mới S3. 3E 80: làm mới, không trả lời.
22	22 DID(2) → 62 DID dữ liệu	Một DID mỗi request. DID lạ → 31; độ dài ≠ 3 → 13.
27	27 01 → 67 01 seed(4); 27 02 key(4) → 67 02	Đã mở khóa: seed = 0. 02 khi chưa có seed → 24. Key sai → 35; sai lần 3 → 36 và chờ 10 s; trong lúc chờ 27 01 → 37. Seed dùng một lần.
2E	2E F1 84 dữ liệu(9) → 6E F1 84	Chỉ F184. Độ dài sai → 13; ngày không phải BCD → 31. Lưu trong RAM đến khi ghi META.
31 FF00	31 01 FF 00 → 71 01 FF 00 00	Chưa ghi F184 → 24. Gửi 0x78, vô hiệu META, xóa sector 4-7, kiểm tra trống. Lỗi → 72.
34	34 00 44 addr(4) size(4) → 74 20 04 02	Định dạng khác 00 / 44 → 31. addr ≠ 0x0801_0000, size = 0, size > 448 KB, không chia hết 4 → 31. Chưa xóa → 70. Đang download → 22.
36	36 BSC dữ liệu → 76 BSC	Không có download → 24. Độ dài < 3 hoặc > 1026 → 13. BSC đúng: ghi, verify, tăng BSC (FF → 00). BSC bằng block trước: trả OK, không ghi lại. BSC khác → 73. Vượt size → 71. Lỗi ghi → 72.
37	37 → 77	Không có download → 24. Chưa nhận đủ size → 24.
31 0202	31 01 02 02 size(4) crc(4) → 71 01 02 02 kq	Gửi 0x78, tính CRC32 trên image. kq = 00 khớp, 01 không khớp. Chưa 37 → 24.
31 FF01	31 01 FF 01 → 71 01 FF 01 kq	Đạt khi 0202 đã khớp và vector table app hợp lý. Đạt → ghi META (size, CRC, version, fingerprint), kq = 00.
28 / 85	28 xx 01 → 68 xx; 85 xx → C5 xx	Bootloader chấp nhận, không làm gì (không có giao tiếp thường hay DTC).

10.6 Data Identifier

DID	Tên	Byte	Nội dung
F180	BootSoftwareIdentification	4	Version bootloader
F181	ApplicationSoftwareIdentification	4	app_version trong META; FFFFFFFF nếu không hợp lệ
F184	ApplicationSoftwareFingerprint	9	Ngày BCD + ID tester
F186	ActiveDiagnosticSession	1	01 hoặc 02
F18C	ECUSerialNumber	12	UID 96 bit của chip
F1A0	(riêng) App status / boot reason	2	Trạng thái META, lý do ở lại bootloader
F1A1	(riêng) Counters	8	prog_count, số frame lỗi CRC

10.7 Routine Identifier

RID	Tên	Ghi chú
FF00	eraseMemory	Định nghĩa trong ISO 14229-1; bootloader luôn xóa toàn bộ vùng APP
0202	checkMemory	Giá trị do OEM định nghĩa; dự án chọn 0202
FF01	checkProgrammingDependencies	Định nghĩa trong ISO 14229-1

10.8 Security Access

- Seed 4 byte lấy từ bộ sinh số ngẫu nhiên phần cứng (RNG) của STM32F407.
- Key = hàm biến đổi đơn giản của seed (XOR với hằng số rồi xoay bit); cùng công thức trong tool PC.
- Tối đa 3 lần sai, sau đó chờ 10 s. Bộ đếm nằm trong RAM (reset xóa bộ đếm – giới hạn được chấp nhận).
- Thuật toán chỉ để minh họa. Sản phẩm thật dùng thuật toán mật mã (ví dụ AES-CMAC) và khóa lưu trong HSM.

11. Drivers

Mọi driver: API đặt tên theo kiểu ST (Tên_Init, Tên_ChứcNăng), trả trạng thái OK/lỗi, không chứa logic nghiệp vụ, mỗi vòng chờ phần cứng có timeout.

Driver	Chức năng	Thanh ghi chính	Điểm cần chú ý
Rcc	Bật/tắt clock và reset từng peripheral; giữ HSI 16 MHz	AHB1ENR, APB1ENR, AHB1RSTR, APB1RSTR, CR, CFGR	Cung cấp hàm reset peripheral dùng khi deinit trước lúc nhảy
Gpio	Cấu hình chân (input, output, AF), đọc, ghi, đảo	MODER, OTyPER, OSPEEDR, PUPDR, AFR, IDR, BSRR	Dùng BSRR để ghi nguyên tử
Systick	Ngắt 1 ms, lấy tick, delay, kiểm tra hết hạn	SysTick CTRL/LOAD/VAL, NVIC	So sánh thời gian bằng phép trừ để đúng khi tràn
Uart	115200 8N1, ngắt RX + ring buffer, gửi, báo gửi xong	USART2 SR, DR, BRR, CR1; NVIC	BRR ở 16 MHz, 115200 = 0x8B; xử lý overrun bằng đọc SR rồi DR
Crc	Reset, tính CRC32 trên mảng byte	CRC DR, CR	Mỗi byte ghi vào DR như một word; đa thức và giá trị đầu cố định của F4
Flash	Unlock/lock, xóa sector, ghi word, đọc lại verify	FLASH KEYR, CR, SR	Chặn sector 0-1; PSize 32 bit; xóa cờ lỗi trước mỗi thao tác; CPU dừng khi xóa
Wdg	Khởi động IWDG ≈ 4 s, refresh	IWDG KR, PR, RLR, SR	Không tắt được sau khi bật; timeout > thời gian xóa một sector
Rng	Lấy số ngẫu nhiên 32 bit	RNG CR, SR, DR	Kiểm tra cờ lỗi seed/clock; cần clock 48 MHz từ PLL48CK
Sys	Reset phần mềm, đọc UID, thao tác VTOR/MSP	SCB, NVIC, vùng UID	Gom các thao tác lỗi Cortex-M
Can (giai đoạn 2)	Khởi tạo 500 kbit/s, bộ lọc, gửi, ngắt RX	CAN MCR, MSR, BTR, TSR, RF0R, FxR	Chân PB8/PB9 AF9

Phụ thuộc cần xử lý: RNG cần nguồn clock 48 MHz từ PLL (PLL48CK). Nếu giữ HSI 16 MHz cho hệ thống, vẫn phải bật PLL chỉ để cấp clock cho RNG, hoặc thay RNG bằng seed tạo từ UID và bộ đếm thời gian. Quyết định khi làm WP7, ghi lại lý do.

12. Application hợp tác

Yêu cầu với app	Lý do
Link tại 0x0801_0000, tối đa 448 KB	Khớp vùng APP
Đặt vector table offset 0x10000 trong cấu hình hệ thống	Ngắt của app không rơi vào vector bootloader
Giảm RAM 16 byte cuối	Không đè BOOT FLAG
Refresh IWDG từ đầu	Bootloader đã bật watchdog
Cùng frame UART và CRC32 với bootloader	Tester nói chuyện với app và bootloader như nhau
Hỗ trợ 10 01/02/03, 11 01, 3E, 22 F181, 28, 85	Đủ cho pre/post-programming
Nhận 10 02: ghi BOOT FLAG, chờ gửi xong, reset; không tự trả 50 02	Bootloader trả lời thay
Sinh file .bin sau khi build; có ít nhất hai phiên bản hành vi khác nhau (tốc độ LED)	Thấy rõ cập nhật đã có hiệu lực

App được phép dùng HAL vì mục tiêu của nó chỉ là minh họa. Toàn bộ năng lực cần thể hiện nằm trong bootloader.

13. Tool phía PC

13.1 Kiến trúc tool

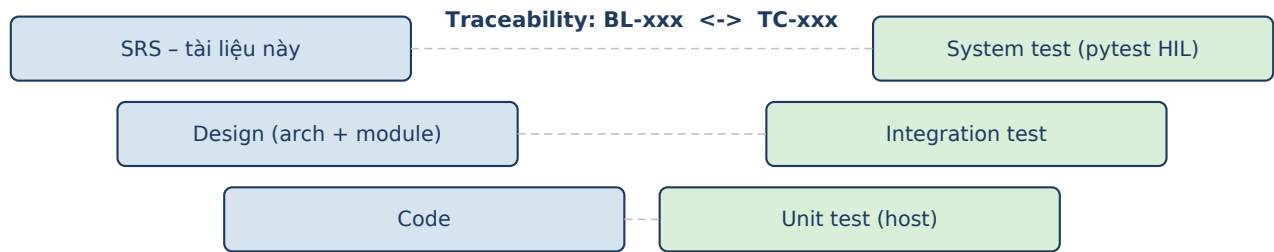
Thành phần	Trách nhiệm
Transport (UartTransport, sau này CanTransport)	Gửi/nhận PDU; với UART thì đóng/mở frame và CRC32
UDS client	Gửi request, chờ response, xử lý 0x78 (chờ tới P2*), chuyển NRC thành lỗi có tên
Image loader	Đọc .bin (và .srec nếu làm thêm), đệm 0xFF cho đủ bội số 4, tính CRC32
Flasher	Chạy đủ flashing sequence 3 pha, gửi 3E khi rảnh, retry block, in tiến độ và thời gian
CLI	Lệnh: flash, read-did, reset, menu thủ công
Test (pytest)	Dùng chung transport và UDS client để kiểm thử trên board

13.2 Tái sử dụng từ STM32_Programmer_V1.py

Phần gốc	Quyết định
Liệt kê và mở cổng serial	Giữ, đưa vào UartTransport
get_crc() (CRC kiểu STM32), word_to_byte()	Giữ nguyên – khớp CRC phần cứng
Menu tương tác	Giữ làm chế độ thủ công; mục menu đổi thành service UDS
Vòng đọc file và gửi theo khối	Viết lại thành vòng TransferData có BSC
Bộ lệnh 0x51–0x5D, ACK 0xA5 / NACK 0x7F	Bỏ, thay bằng UDS
Biến toàn cục, tên file cố định, timeout 2 s cố định	Thay bằng đối tượng, tham số dòng lệnh, P2/P2*

Bản gốc được giữ nguyên trong thư mục legacy để thấy rõ phần tái sử dụng.

14. Kiểm chứng



Mỗi milestone đi đủ vòng: cập nhật SRS -> design -> code -> unit test -> integration -> tag

Hình 14.1 – Các mức kiểm chứng

14.1 Chiến lược

Mức	Đối tượng	Cách làm
Kiểm tra driver	Từng driver	Chương trình test nhỏ trên board, quan sát bằng debugger, LED, logic analyzer
Unit test trên PC (tùy chọn)	UartTp, Dcm, Meta	Build bằng gcc trên PC với driver giả; chạy nhanh, lặp lại được
Kiểm thử hệ thống (HIL)	Bootloader + app trên board	pytest qua tool PC; chạy toàn bộ sau mỗi WP
Kiểm thử mất nguồn	Toàn hệ thống	Rút nguồn thật tại từng bước + reset có chủ đích trong bản debug
Đo đạc	Hiệu năng	Kích thước, thời gian boot, thời gian xóa, thông lượng

14.2 Danh sách test hệ thống

ID	Kịch bản	Kỳ vọng	Truy vết
TC-01	Reset, app hợp lệ, không nhấn nút	Chạy app	FR-01, FR-02
TC-02	Giữ PA0 khi reset	Ở lại bootloader	FR-01
TC-03	Xóa app rồi reset	Ở lại bootloader	FR-02
TC-04	Frame sai CRC	Không trả lời	FR-04
TC-05	Nửa frame, chờ 100 ms, frame đúng	Frame đúng được xử lý	FR-04
TC-06	SID 19	7F 19 11	FR-05
TC-07	34 ở Default	7F 34 7F	FR-05
TC-08	SID 19 functional	Không trả lời	FR-05
TC-09	3E 80	Không trả lời, S3 làm mới	FR-05
TC-10	10 02	50 02 00 32 01 F4	FR-05
TC-11	Im lặng 6 s ở Programming	22 F186 trả 01	FR-11
TC-12	27 02 khi chưa 27 01	7F 27 24	FR-06
TC-13	Key sai 3 lần	35, 35, 36; 27 01 trong 10 s → 37	FR-14
TC-14	31 FF00 khi chưa mở khóa	7F 31 33	FR-06
TC-15	31 FF00 khi chưa ghi F184	7F 31 24	FR-08, FR-13
TC-16	34 địa chỉ 0x0800_0000	7F 34 31	FR-07
TC-17	34 trước khi xóa	7F 34 70	FR-08
TC-18	36 khi chưa 34	7F 36 24	FR-08
TC-19	36 BSC nhảy cóc	7F 36 73	FR-05
TC-20	36 gửi lại block trước	76 BSC, không ghi lại	FR-05

ID	Kịch bản	Kỳ vọng	Truy vết
TC-21	Image > 256 block	BSC quay vòng đúng, nạp thành công	FR-05
TC-22	36 vượt size	7F 36 71	FR-05
TC-23	37 khi thiếu dữ liệu	7F 37 24	FR-08
TC-24	31 0202 CRC sai	kq 01; FF01 kq 01; reset vẫn ở bootloader	FR-09
TC-25	Flash v1 → v2 → v1	Đúng phiên bản chạy sau mỗi lần	FR-05, SC-1
TC-26	App đang chạy, tester gửi 10 02	Vào bootloader, nhận 50 02	FR-10
TC-27	Mất nguồn ở 6 thời điểm (mục 8.6)	Như bảng ở mục 8.6	NFR-05, SC-2
TC-28	Chương trình treo có chủ đích (bản debug)	IWDG reset	NFR-08
TC-29	22 F180 / F186 / F18C	Đúng độ dài và nội dung	FR-12

14.3 Đo đặc cần ghi lại

Chỉ số	Cách đo	Mục tiêu
Kích thước bootloader	Map file	≤ 32 KB
Thời gian reset → app	GPIO toggle + logic analyzer	≤ 500 ms
Thời gian xóa sector 128 KB	Logic analyzer	Ghi lại, < P2*
Thông lượng nạp	Tool PC	≥ 9 KB/s ở 115200
Thời gian nạp toàn trình	Tool PC	Ghi lại

15. Cấu trúc project

15.1 Cây thư mục đầy đủ

Một repository chứa cả bootloader, app, tool PC, test và tài liệu. Cây dưới đây là trạng thái khi hoàn tất giai đoạn 2; các file đánh dấu "giai đoạn 2" chưa cần tạo ở giai đoạn 1.

Cây thư mục

```
stm32f407-uds-bootloader/
├── README.md           tổng quan, cách build/ nạp, kết quả, video
├── .gitignore          bỏ Debug/ Release/ .settings/ *.launch __pycache__/
├── LICENSE
├── common/             file dùng chung bootloader + app
│   ├── fbl_memmap.h    địa chỉ BOOT, META, APP, BOOT FLAG, kích thước
│   ├── fbl_meta.h      bố cục bản ghi META
│   ├── fbl_bootflag.h  bố cục BOOT FLAG, giá trị magic
│   └── fbl_protocol.h  SOF, ADDR, PDU tối đa, SID, NRC
├── bootloader/         project STM32CubeIDE (Empty + CMSIS)
│   ├── STM32F407VGTX_FLASH.ld  FLASH 32 KB, RAM 128 KB - 16 B
│   ├── Startup/
│   │   └── startup_stm32f407vgtx.s
│   ├── Src/
│   │   ├── main.c      khởi tạo + vòng lặp chính
│   │   ├── syscalls.c, sysmem.c  do IDE tạo
│   ├── Config/
│   │   ├── Bl_Cfg.h    baud, timing P2/P2*/S3, hằng số chung
│   │   ├── Bl_Version.h  version bootloader (DID F180)
│   │   ├── Dcm_Cfg.c / .h  bảng service, DID, RID
│   │   └── PduR_Cfg.h   chọn transport (UART / CAN)
│   ├── Common/
│   │   ├── Std_Types.h  kiểu trả về OK/lỗi, bool, NULL
│   │   └── Bl_Util.c / .h  đọc/ghi big-endian, so sánh an toàn
│   ├── Drivers/
│   │   ├── CMSIS/      copy từ STM32Cube_FW_F4
│   │   │   ├── Include/  core_cm4.h ...
│   │   │   └── Device/ST/STM32F4xx/Include/  stm32f4xx.h, stm32f407xx.h
│   │   └── BL/
│   │       ├── Rcc.c / .h
│   │       ├── Gpio.c / .h
│   │       ├── SysTick.c / .h
│   │       ├── Uart.c / .h
│   │       ├── Crc.c / .h
│   │       ├── Flash.c / .h
│   │       ├── wdg.c / .h
│   │       ├── Rng.c / .h
│   │       ├── Sys.c / .h
│   │       └── Can.c / .h  (giai đoạn 2)
│   ├── Transport/
│   │   ├── PduR.c / .h
│   │   ├── UartTp.c / .h
│   │   ├── CanIf.c / .h  (giai đoạn 2)
│   │   └── CanTp.c / .h  (giai đoạn 2)
│   ├── Diagnostics/
│   │   ├── Dcm.c / .h    nhận PDU, dispatch, NRC, session, timing
│   │   ├── Dcm_Svc.c / .h  xử lý từng service
│   │   └── Dcm_Prog.c / .h  state machine lập trình
│   └── Services/
│       ├── BootMgr.c / .h
│       ├── FlashMgr.c / .h
│       ├── Meta.c / .h
│       └── SecM.c / .h
```

Cây thư mục (tiếp)

app/	project STM32CubeIDE (STM32Cube + HAL)
app.ioc	cấu hình CubeMX
STM32F407VGTX_FLASH.ld	ORIGIN 0x0801_0000, 448 KB; RAM - 16 B
Core/Inc/ Core/Src/	
main.c	LED, vòng lặp, gọi App_Uds
system_stm32f4xx.c	VECT_TAB_OFFSET = 0x10000
stm32f4xx_it.c	ngắt USART2
app_tp.c / .h	frame UART + CRC32 (giống bootloader)
app_uds.c / .h	mini-Dcm: 10, 11, 22 F181, 28, 3E, 85
app_version.h	version app
Drivers/	HAL + CMSIS do CubeMX sinh
host/	tool Python
requirements.txt	pyserial, pytest (sau: python-can, can-isotp)
fbl_cli.py	lệnh: flash, read-did, reset, menu
config.py	cổng, baud, timing, key
fbl/	
__init__.py	
crc.py	get_crc() kiểu STM32 (từ bản gốc)
transport.py	Transport, UartTransport (sau: CanTransport)
uds.py	UdsClient, NRC → exception, xử lý 0x78
security.py	tính key từ seed
image.py	đọc .bin, đệm 0xFF, tính CRC
flasher.py	flashing sequence 3 pha
legacy/	
STM32_Programmer_V1.py	bản gốc niekiran, không sửa
test/	kiểm thử trên board (pytest)
conftest.py	fixture: mở cổng, đưa board về trạng thái biết trước
test_transport.py	TC-04, TC-05
test_session.py	TC-06 ... TC-11, TC-29
test_security.py	TC-12 ... TC-14
test_programming.py	TC-15 ... TC-25
test_app_entry.py	TC-01 ... TC-03, TC-26
test_power_loss.py	TC-27, TC-28
images/	app_v1.bin, app_v2.bin, app_bad_msp.bin
tools/	
size_check.py	đọc map file, báo lỗi nếu bootloader > 32 KB
collect_images.py	copy .bin của app vào test/images
docs/	
SAD_v3.1.pdf	tài liệu này
diagrams/	hình memory map, boot flow, sequence
test_report.md	kết quả mỗi lần chạy test
measurements.md	số đo: kích thước, thời gian, tốc độ
journal.md	nhật ký theo tuần
interview_notes.md	câu hỏi và ý trả lời

15.2 Vai trò từng thư mục

Thư mục	Vai trò	Quy tắc
common/	Hằng số và bố cục dữ liệu mà bootloader và app phải thống nhất (địa chỉ, META, BOOT FLAG, định dạng frame)	Chỉ chứa header; cả hai project thêm đường dẫn include tới đây; sửa ở đây là sửa cho cả hai
bootloader/Config/	Mọi tham số có thể thay đổi	Không có số ma thuật ở thư mục khác
bootloader/Common/	Kiểu dữ liệu và tiện ích không phụ thuộc phần cứng	Không include header chip
bootloader/Drivers/BI/	Driver thanh ghi	Chỉ nơi này include stm32f4xx.h (cùng Sys cho lỗi Cortex)
bootloader/Transport/	PduR và transport	Không biết UDS
bootloader/Diagnostics/	UDS	Không biết UART/CAN
bootloader/Services/	Chức năng bootloader	Gọi Drivers qua API

Thư mục	Vai trò	Quy tắc
app/	Demo app hợp tác	Được dùng HAL; app_tp và app_uds dùng lại định dạng trong common/
host/	Tool PC	Chỉ transport.py dùng pyserial/python-can
test/	Kiểm thử hệ thống	Mỗi file test ứng với một nhóm TC
tools/	Script hỗ trợ build và kiểm tra	Chạy được độc lập
docs/	Tài liệu và bằng chứng	Cập nhật cuối mỗi WP

15.3 File nào được tạo ở work package nào

WP	File / thư mục tạo mới
WP0	README.md, .gitignore, common/fbl_memmap.h, bootloader/ (khung, CMSIS, linker), app/ (khung, linker)
WP1	Services/BootMgr, common/fbl_bootflag.h, phần đầu Sys
WP2	Drivers/BI/Rcc, Gpio, Systick, Uart, Crc, Flash, Wdg, Rng, Sys; Common/Std_Types.h
WP3	Transport/PduR, UartTp; Config/PduR_Cfg.h; common/fbl_protocol.h; host/fbl/crc.py, transport.py
WP4	Diagnostics/Dcm, Dcm_Svc (10, 11, 22, 3E); Config/Dcm_Cfg, BI_Cfg.h, BI_Version.h; host/fbl/uds.py; test/conftest.py, test_session.py, test_transport.py
WP5	Services/FlashMgr; Diagnostics/Dcm_Prog; Dcm_Svc (31 FF00, 34, 36, 37); host/fbl/image.py, flasher.py, fbl_cli.py; test_programming.py
WP6	Services/Meta; common/fbl_meta.h; Dcm_Svc (2E, 31 0202, 31 FF01)
WP7	Services/SecM; Dcm_Svc (27); host/fbl/security.py; test_security.py
WP8	app/Core/Src/app_tp, app_uds, app_version.h; test_app_entry.py
WP9	test_power_loss.py; tools/; docs/ đầy đủ
WP10	Drivers/BI/Can; Transport/CanIf, CanTp; host transport CanTransport

15.4 Chia sẻ common/ trong STM32CubeIDE

1. Ở mỗi project: Properties → C/C++ General → Paths and Symbols → Includes → thêm đường dẫn tương đối tới thư mục common/ của repository.
2. Nếu muốn thấy file trong cây project: New → Folder → Advanced → Link to alternate location → trở tới common/.
3. Không copy common/ vào từng project – hai bản sao sẽ lệch nhau.

16. Quy trình phát triển

16.1 Git

- Mỗi work package một nhánh (ví dụ wp3-uart-transport). Merge vào main khi đạt tiêu chí hoàn thành, gắn tag.
- Commit nhỏ, mỗi commit build được; message dạng loại(phạm vi): mô tả, ví dụ feat(flash): erase sector.
- Không commit thư mục build của IDE.

16.2 Quy tắc mã nguồn

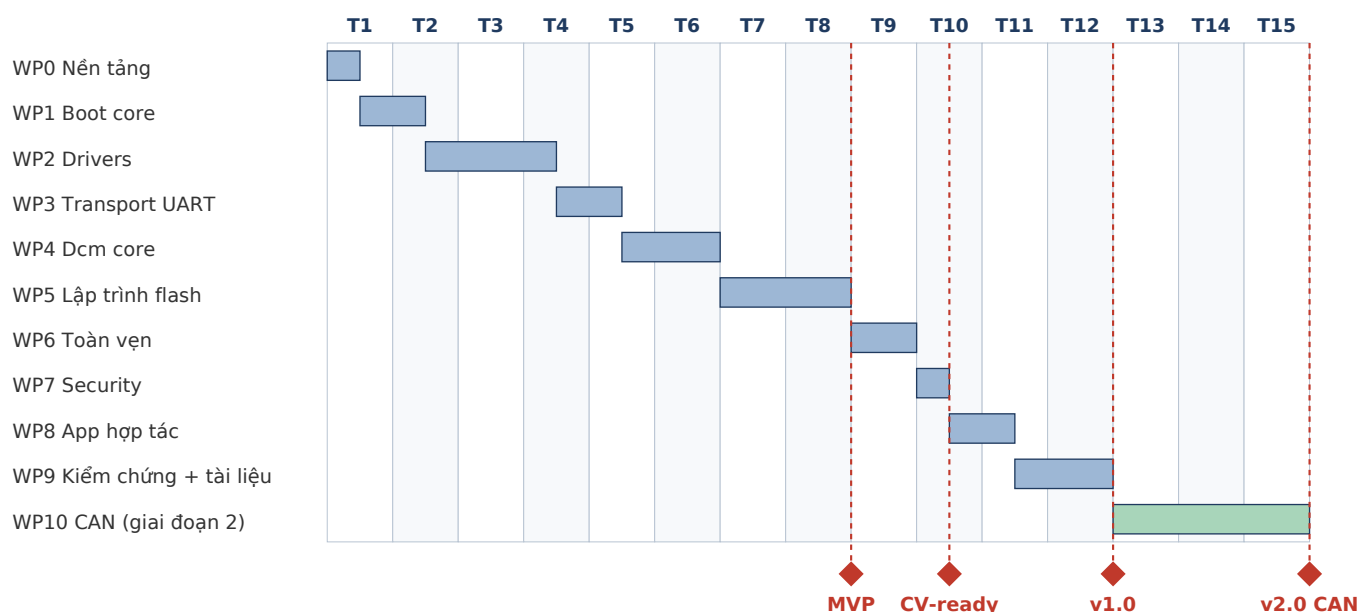
- C99, bật cảnh báo đầy đủ và coi cảnh báo là lỗi; tối ưu kích thước cho bootloader.
- Kiểu dữ liệu cố định độ rộng; không số ma thuật ngoài file cấu hình.
- Mỗi hàm công khai có chú thích: làm gì, tham số, giá trị trả về, yêu cầu liên quan.
- Tham khảo MISRA C:2012 ở mức hợp lý (không ép kiểu con trỏ tùy tiện, mọi switch có default, không goto).

16.3 Tiêu chí hoàn thành chung cho mỗi work package

1. Build không cảnh báo.
2. Tiêu chí riêng của WP đạt, có bằng chứng (log, hình, số đo).
3. Test của các WP trước vẫn pass.
4. Tài liệu được cập nhật nếu thiết kế thay đổi.
5. Ghi 3 dòng vào nhật ký: đã làm gì, vướng gì, giải quyết thế nào.

17. Kế hoạch triển khai và timeline

17.1 Timeline



Hình 16.1 – Timeline 15 tuần (T1 bắt đầu thứ Hai 28/09/2026)

Mốc	Tuần	Ngày dự kiến	Ý nghĩa
MVP	Hết T8	22/11/2026	Nạp được app qua UDS
CV-ready	Giữa T10	02/12/2026	Có toàn vẹn + security; đủ để ghi vào CV và nói khi phỏng vấn
v1.0	Hết T12	20/12/2026	Hoàn chỉnh giai đoạn UART: app hợp tác, test, tài liệu
v2.0	Hết T15	10/01/2027	Chạy trên CAN, cùng bộ test

Timeline giả định khoảng 10–12 giờ mỗi tuần. Nếu chậm, giữ thứ tự và dời mốc; không bỏ bước kiểm chứng.

17.2 Work packages

WP0 – Nền tảng	28/09 – 01/10 (0.5 tuần)
Công việc	- Tạo repo và cấu trúc thư mục. - Project bootloader (Empty + CMSIS), project app (HAL). - Linker: bootloader 32 KB tại 0x0800_0000; app tại 0x0801_0000, 448 KB; app đặt vector table offset. - Giảm RAM 16 byte cuối ở cả hai project.
Hoàn thành khi	Hai project build; app nạp độc lập ở địa chỉ mới chạy được sau khi bootloader nhảy thử bằng debugger.
Sản phẩm	Repo, hai project, linker script
WP1 – Boot core	02/10 – 07/10 (1 tuần)
Công việc	- BootMgr: đọc BOOT FLAG, đọc nút, kiểm tra MSP/reset vector. - Trình tự deinit và nhảy (8.2). - Tạm thời chưa có META: app coi là hợp lệ nếu vector table hợp lý.
Hoàn thành khi	Reset → app chạy (ngắt của app hoạt động); giữ nút → ở lại bootloader.
Sản phẩm	BootMgr, Sys (một phần)

WP2 - Drivers	08/10 - 21/10 (2 tuần)
Công việc	- Rcc, Gpio, Systick, Uart (ngắt + ring buffer), Crc, Flash, Wdg, Rng, Sys. - Mỗi driver một chương trình test nhỏ.
Hoàn thành khi	Uart echo file 10 KB không mất byte; CRC("123456789") = 0x1556F485; xóa/ghi sector 7 đúng, sector 0 bị từ chối; IWDG reset khi cố ý treo.
Sản phẩm	9 driver + ghi chú test
WP3 - Transport UART	22/10 - 29/10 (1 tuần)
Công việc	- UartTp: nhận/gửi frame, timeout 50 ms. - PduR với giao diện ở 7.4. - Tool PC: UartTransport tái sử dụng serial + get_crc().
Hoàn thành khi	PC gửi 3E 00 nhận 7E 00; frame sai CRC không có trả lời; 100 frame liên tiếp không mất.
Sản phẩm	UartTp, PduR, host transport
WP4 - Dcm core	30/10 - 08/11 (1.5 tuần)
Công việc	- Bảng service, thứ tự NRC, session, P2/P2*/S3, 0x78, SPRMIB, functional. - Service 10, 11, 22, 3E. - UDS client phía PC xử lý 0x78 và NRC.
Hoàn thành khi	TC-06 ... TC-11, TC-29 pass.
Sản phẩm	Dcm, Dcm_Svc (một phần)
WP5 - Lập trình flash	09/11 - 22/11 (2 tuần)
Công việc	- FlashMgr, Dcm_Prog. - Service 31 FF00, 34, 36, 37 (tạm thời chưa bắt buộc 27 và 2E). - Flasher PC: đọc .bin, đếm, gửi theo khối, in tiến độ.
Hoàn thành khi	Nạp app v1 → v2 bằng tool; TC-16 ... TC-23 pass. Mốc MVP.
Sản phẩm	FlashMgr, Dcm_Prog, flasher
WP6 - Toàn vẹn	23/11 - 29/11 (1 tuần)
Công việc	- Meta: đọc, vô hiệu hóa, ghi. 2E F184, 31 0202, 31 FF01. - BootMgr dùng META + CRC để quyết định.
Hoàn thành khi	TC-01 ... TC-03, TC-15, TC-24 pass; rút cáp giữa lúc 36 → ở lại bootloader, nạp lại được.
Sản phẩm	Meta, bổ sung Dcm_Svc
WP7 - Security	30/11 - 02/12 (0.5 tuần)
Công việc	- SecM: seed, key, bộ đếm, thời gian chờ. 27; bắt buộc mở khóa cho 2E/31/34/36/37. - Quyết định nguồn seed (RNG hay UID + timer).
Hoàn thành khi	TC-12 ... TC-14 pass. Mốc CV-ready.
Sản phẩm	SecM, cập nhật bảng service
WP8 - App hợp tác	03/12 - 10/12 (1 tuần)
Công việc	- App: frame + CRC32, 10 01/02/03, 11, 3E, 22 F181, 28, 85; BOOT FLAG. - Bootloader trả 50 02 thay app. - Flasher chạy đủ 3 pha.
Hoàn thành khi	TC-25, TC-26 pass: nạp hoàn toàn không chạm board.
Sản phẩm	App v1, v2

WP9 – Kiểm chứng và tài liệu	11/12 – 20/12 (1.5 tuần)
Công việc	• Hoàn thiện bộ pytest; test mất nguồn; đo đặc. • README: sơ đồ, trình tự, số đo, hình logic analyzer, video. • Rà soát mã, chú thích, file cấu hình.
Hoàn thành khi	Toàn bộ test pass; số đo trong README; một người khác nạp được trong 15 phút. Mốc v1.0.
Sản phẩm	Báo cáo test, README, video
WP10 – CAN	21/12/2026 – 10/01/2027 (3 tuần)
Công việc	• Phần cứng CAN; driver Can; CanIf; CanTp (ISO 15765-2). • Đổi cấu hình PduR. • Tool PC: CanTransport; chạy lại toàn bộ test.
Hoàn thành khi	Toàn bộ test pass trên CAN; Dcm và Services không đổi mã nguồn. Mốc v2.0.
Sản phẩm	Can, CanIf, CanTp

18. Phân rã công việc (WBS)

18.1 Nguyên tắc chia từ lớn đến nhỏ

Cấp	Tên	Ví dụ	Kích thước
1	Work package (WP)	WP2 – Drivers	0.5–3 tuần, có mốc và tiêu chí hoàn thành (Chương 17)
2	Hạng mục	2.4 – Uart	Một module hoặc một nhóm chức năng
3	Task	2.4.3 – Ngắt RX + ring buffer	≤ 4 giờ, có kết quả kiểm tra được ngay

- Một task không làm xong trong một buổi tối thì chia tiếp.
- Mỗi task có cột **Kiểm tra**: dấu hiệu cụ thể để biết đã xong (LED, số đo, test case). Không có thì chưa phải task.
- Làm theo thứ tự ID trong WP; chỉ sang WP sau khi mọi task của WP trước xong.
- Theo dõi bằng GitHub Projects: mỗi task cấp 3 là một issue, gắn nhãn WP; cột Todo → Doing → Done.
- Commit tham chiếu ID task, ví dụ feat(uart): rx irq + ring buffer (2.4.3).

18.2 Tổng hợp

WP	Tên	Số task	Ước lượng	Tuần
WP0	Nền tảng	7	6 h	T1
WP1	Boot core	6	10 h	T1-T2
WP2	Drivers	16	27 h	T2-T4
WP3	Transport UART	6	10.5 h	T4-T5
WP4	Dcm core	7	15 h	T5-T6
WP5	Lập trình flash	9	19 h	T7-T8
WP6	Toàn vẹn	6	10 h	T9
WP7	Security	4	5 h	T10
WP8	App hợp tác	6	10 h	T10-T11
WP9	Kiểm chứng và bàn giao	6	15 h	T11-T12
WP10	CAN	8	30 h	T13-T15
	Tổng		157.5 h	15 tuần

Giai đoạn UART (WP0-WP9) khoảng 130 giờ, tức khoảng 11 giờ mỗi tuần trong 12 tuần. Ước lượng dùng để lập kế hoạch; ghi thời gian thực tế vào nhật ký để lần sau ước lượng tốt hơn.

18.3 WP0 - Nền tảng (≈ 6 h)

ID	Task	Giờ	Kiểm tra (xong khi)
0.1	Repository		
0.1.1	Tạo repo, .gitignore, README khung, tạo các thư mục cấp 1	1	Repo trên GitHub
0.2	Project bootloader		
0.2.1	Tạo project Empty cho STM32F407G-DISC1	0.5	Project mở được
0.2.2	Copy CMSIS, thêm include path, define STM32F407xx	1	Build với stm32f4xx.h
0.2.3	Sửa linker: FLASH 32 KB, RAM trừ 16 B	0.5	Map file đúng vùng
0.3	Project app		
0.3.1	Tạo project STM32Cube: LED PD12-15, USART2 có ngắt	1	Blinky chạy
0.3.2	Linker ORIGIN 0x0801_0000 / 448 KB, RAM trừ 16 B, VECT_TAB_OFFSET, xuất .bin	1	Có app.bin

ID	Task	Giờ	Kiểm tra (xong khi)
0.4	Common		
0.4.1	Tạo common/fbl_memmap.h; thêm include path ở hai project	1	Cả hai build

18.4 WP1 - Boot core (≈ 10 h)

ID	Task	Giờ	Kiểm tra (xong khi)
1.1	Nhảy sang app		
1.1.1	Đọc MSP và reset vector của app, kiểm tra phạm vi	1	Giá trị đúng trên debugger
1.1.2	Trình tự deinit: ngắt, SysTick, NVIC, reset peripheral	2	Review theo 8.2
1.1.3	Đặt VTOR, MSP, nhảy; kiểm tra ngắt của app chạy	2	LED app nháy bằng SysTick
1.2	Quyết định boot		
1.2.1	Đọc PA0 có chống dội 20 ms	1	Giữ nút → ở lại
1.2.2	BOOT FLAG: định nghĩa trong common, đọc và xóa	2	Ghi flag bằng debugger → ở lại
1.2.3	Luồng quyết định đầy đủ + LED trạng thái	2	Đủ 3 nhánh của Hình 8.1

18.5 WP2 - Drivers (≈ 27 h)

ID	Task	Giờ	Kiểm tra (xong khi)
2.1	Rcc		
2.1.1	Bật clock, reset peripheral	2	Dùng được ở Gpio
2.2	Gpio		
2.2.1	Init (input/output/AF), Read, Write, Toggle	2	LED, nút chạy
2.2.2	Cấu hình AF7 cho PA2/PA3	1	Đo được mức trên chân
2.3	Systick		
2.3.1	Ngắt 1 ms, GetTick, kiểm tra hết hạn chịu tràn	2	LED 1 Hz đo bằng LA
2.4	Uart		
2.4.1	Init 115200 8N1 (BRR)	2	Terminal nhận chuỗi
2.4.2	Gửi tuần tự + báo gửi xong (cờ TC)	1	Chuỗi đúng
2.4.3	Ngắt RX + ring buffer 512 B + xử lý overrun	3	Không mất byte
2.4.4	Test echo file 10 KB	1	So sánh file khớp
2.5	Crc		
2.5.1	Init, Calculate (byte nạp như word)	2	"123456789" → 0x1556F485
2.6	Flash		
2.6.1	Unlock/lock, xóa cờ lỗi, chờ BSY có timeout	2	Không treo
2.6.2	EraseSector, chặn sector 0-1	2	Sector 7 trống; sector 0 bị từ chối
2.6.3	ProgramWord + đọc lại verify	2	Memory view đúng
2.6.4	Đo thời gian xóa sector 16/64/128 KB	1	Ghi measurements.md
2.7	Wdg		
2.7.1	IWDG ≈ 4 s, refresh; test treo có chủ đích	1.5	Board tự reset
2.8	Rng		
2.8.1	Quyết định nguồn clock 48 MHz hoặc phương án thay thế; implement	1.5	Hai lần đọc khác nhau
2.9	Sys		
2.9.1	Reset phần mềm, đọc UID, gom thao tác VTOR/MSP từ BootMgr	1	UID đúng với ST-LINK

18.6 WP3 – Transport UART (≈ 10.5 h)

ID	Task	Giờ	Kiểm tra (xong khi)
3.1	PduR		
3.1.1	Khai báo interface, cấu hình route tới UartTp	2	Build, review 7.4
3.2	UartTp		
3.2.1	State machine nhận frame (SOF, ADDR, LEN, DATA, CRC)	3	Frame đúng → RxIndication
3.2.2	Gửi frame + TxConfirmation	1.5	PC nhận frame đúng CRC
3.2.3	Timeout 50 ms giữa byte, bộ đếm lỗi	1	Nửa frame bị bỏ
3.3	Host		
3.3.1	crc.py (get_crc từ bản gốc), transport.py (UartTransport)	2	Gửi/nhận frame
3.3.2	Test: 3E lặp 100 lần, CRC sai, nửa frame	1	TC-04, TC-05

18.7 WP4 – Dcm core (≈ 15 h)

ID	Task	Giờ	Kiểm tra (xong khi)
4.1	Khung		
4.1.1	Nhận PDU, buffer response, gửi qua PduR	2	Echo SID + 0x40
4.1.2	Bảng service + thứ tự kiểm tra NRC	3	TC-06, TC-07
4.2	Session và timing		
4.2.1	Session 01/02, khóa security khi đổi session	2	TC-10
4.2.2	P2 / P2* (gửi 0x78), S3	3	TC-11; LA thấy 0x78
4.2.3	SPRMIB, functional addressing	1.5	TC-08, TC-09
4.3	Service cơ bản		
4.3.1	11 01, 3E 00, 22 F180/F186/F18C	2	TC-29
4.4	Host		
4.4.1	uds.py: request, chờ 0x78, NRC thành exception	1.5	Script in tên NRC

18.8 WP5 – Lập trình flash (≈ 20 h)

ID	Task	Giờ	Kiểm tra (xong khi)
5.1	Services		
5.1.1	FlashMgr: kiểm tra vùng APP, ghi nhớ sector đã xóa	2	Unit kiểm bằng debugger
5.2	Dcm_Prog		
5.2.1	State machine và bảng chuyển trạng thái (Hình 8.2)	3	Sai thứ tự → 0x24
5.3	Service		
5.3.1	31 01 FF00: 0x78, xóa từng sector	3	Sector 4-7 trống
5.3.2	34: kiểm tra DFI, ALFID, địa chỉ, kích thước	2	TC-16, TC-17
5.3.3	36: BSC, block lặp, ghi, verify, vượt size	3	TC-18 ... TC-22
5.3.4	37	1	TC-23
5.4	Host		
5.4.1	image.py: đọc .bin, đếm, CRC	1	Kích thước chia hết 4
5.4.2	flasher.py bản MVP + fbl_cli flash	2	Nạp app v1 → v2
5.4.3	Test BSC quay vòng (> 256 block)	2	TC-21 — mốc MVP

18.9 WP6 – Toàn vẹn (≈ 10 h)

ID	Task	Giờ	Kiểm tra (xong khi)
6.1	Meta		
6.1.1	fbl_meta.h; Read, Invalidate, Write, meta_crc	3	Đọc lại đúng sau reset
6.2	Service		
6.2.1	2E F184: kiểm BCD, giữ trong RAM	1.5	TC-15
6.2.2	31 0202: CRC image, gửi 0x78	2	TC-24
6.2.3	31 FF01: kiểm vector table, ghi META	1.5	22 F181 trả version
6.3	BootMgr		
6.3.1	Dùng META + CRC để quyết định chạy app	1	TC-01 ... TC-03
6.4	Kiểm tra		
6.4.1	Rút cáp giữa lúc 36, reset, nạp lại	1	Ở lại bootloader

18.10 WP7 – Security (≈ 5 h)

ID	Task	Giờ	Kiểm tra (xong khi)
7.1	SecM		
7.1.1	Seed (RNG hoặc thay thế), thuật toán key, seed dùng một lần	2	Seed đổi mỗi lần
7.1.2	Đếm số lần sai, chờ 10 s	1	TC-13
7.2	Dcm		
7.2.1	Service 27; bật yêu cầu mở khóa cho 2E/31/34/36/37	1	TC-12, TC-14
7.3	Host		
7.3.1	security.py, thêm bước 27 vào flasher	1	Mốc CV-ready

18.11 WP8 – App hợp tác (≈ 10 h)

ID	Task	Giờ	Kiểm tra (xong khi)
8.1	App		
8.1.1	app_tp: frame + CRC32 giống bootloader	2	3E tới app có trả lời
8.1.2	app_uds: 10 01/02/03, 11, 3E, 22 F181	3	Đọc được version app
8.1.3	28, 85 trong app	1	Positive response
8.1.4	10 02 → ghi BOOT FLAG, chờ gửi xong, reset	1	Board vào bootloader
8.2	Bootloader		
8.2.1	Trả 50 02 thay app khi thấy BOOT FLAG	1	TC-26
8.3	Host		
8.3.1	Flasher đủ 3 pha; build app v1, v2, app lỗi MSP	2	TC-25

18.12 WP9 - Kiểm chứng và bàn giao (≈ 15 h)

ID	Task	Giờ	Kiểm tra (xong khi)
9.1	Test		
9.1.1	confptest.py: fixture cổng, đưa board về trạng thái biết trước	2	Chạy lặp không lỗi
9.1.2	Hoàn thiện test_session, security, programming, app_entry	4	Đủ TC-01 ... TC-26, 29
9.1.3	Test mất nguồn: rút thật 6 điểm + bản debug reset có chủ đích	3	TC-27, TC-28
9.2	Đo đạc		
9.2.1	5 chỉ số ở 14.3	2	measurements.md
9.3	Tài liệu		
9.3.1	README: sơ đồ, trình tự, số đo, hình logic analyzer, video 1-2 phút	3	Mốc v1.0
9.3.2	Rà soát mã, chú thích, cấu hình	1	Build không cảnh báo

18.13 WP10 - CAN (≈ 30 h)

ID	Task	Giờ	Kiểm tra (xong khi)
10.1	Phần cứng		
10.1.1	Đấu SN65HVD230, adapter, điện trở 120 Ω	2	candump thấy frame
10.2	Can		
10.2.1	Init, bit timing 500 kbit/s, bộ lọc, gửi, ngắt RX	8	Loopback và bus thật
10.3	CanIf		
10.3.1	Chuyển frame giữa Can và CanTp theo CAN ID	2	Route đúng
10.4	CanTp		
10.4.1	Nhận SF, FF, CF; gửi FC	6	Nhận PDU 1026 B
10.4.2	Gửi SF, FF, chờ FC, CF, STmin	5	Gửi PDU dài
10.4.3	Timeout N_As, N_Bs, N_Cr	2	Frame thiếu bị hủy
10.5	Tích hợp		
10.5.1	Đổi PduR_Cfg sang CanTp; host CanTransport	2	3E qua CAN
10.5.2	Chạy lại toàn bộ test, sửa lỗi	3	Mốc v2.0

19. Rủi ro

Rủi ro	Ảnh hưởng	Giảm thiểu
Nhảy sang app lỗi ngẫu nhiên (quên VTOR, còn ngắt pending)	Cao	Làm đúng trình tự 8.2; test sớm ở WP1
Lỗi xóa bootloader khi thử flash	Cao	Flash driver chặn sector 0-1 từ WP2; có thể nạp lại bằng ST-LINK
Tester timeout khi xóa flash	Trung bình	Gửi 0x78 trước mỗi thao tác dài; PC chờ P2* sau 0x78
CRC PC và MCU không khớp	Trung bình	Dùng giá trị kiểm tra "123456789" ngay từ WP2
Mất byte UART	Trung bình	Ring buffer + xử lý overrun; test 10 KB ở WP2
RNG cần PLL48CK	Thấp	Quyết định ở WP7 (bật PLL cho RNG hoặc seed từ UID + timer)
Chậm tiến độ	Trung bình	Giữ thứ tự WP; mốc CV-ready ưu tiên hơn v1.0
Board không có phần cứng CAN phía PC	Thấp	Mua trước WP10; giai đoạn 1 không phụ thuộc

20. Chuẩn bị phỏng vấn

Câu hỏi	Ý chính (từ chính dự án)
Bootloader chuyển sang app thế nào?	Trình tự 8 bước ở 8.2; vì sao phải xóa NVIC pending và đặt VTOR
Mất nguồn khi đang nạp thì sao?	META xóa đầu tiên, ghi cuối cùng; bảng 8.6; vì sao chọn single slot thay A/B
Flashing sequence UDS?	3 pha, 14 bước; ý nghĩa từng service
NRC 0x24, 0x73, 0x78 khi nào?	Sai thứ tự; BSC sai; đang xóa flash
P2, P2*, S3 là gì?	50 ms, 5000 ms, 5000 ms; response 50 02 00 32 01 F4
Vì sao CPU dừng khi xóa flash?	F407 một bank: không đọc lệnh được khi flash đang bận
Security access hoạt động ra sao?	Seed/key, giới hạn số lần, thời gian chờ; bản thật dùng AES-CMAC + HSM
Chuyển sang CAN tốn bao nhiêu công?	Chỉ Can + CanIf + CanTp + cấu hình PduR; Dcm không đổi
CRC32 STM32 khác CRC-32 chuẩn?	Cùng đa thức, nạp theo word, không đảo bit, không XOR cuối
Ngắt và main chia sẻ dữ liệu ra sao?	Ring buffer: mỗi bên sở hữu một con trỏ, volatile

Khi trình bày, nói rõ phần học từ khóa niekiran (ý tưởng lệnh, tool Python) và phần tự thiết kế (driver thanh ghi, UDS, META, transport tách lớp, bộ test). Mang theo số đo thật.

21. Phụ lục

21.1 Phụ lục A – Bảng NRC

NRC	Tên	Dùng khi
11	serviceNotSupported	SID không có
12	subFunctionNotSupported	Sub-function không có
13	incorrectMessageLengthOrInvalidFormat	Sai độ dài/định dạng
21	busyRepeatRequest	Đang xử lý request khác
22	conditionsNotCorrect	34 khi đang download
24	requestSequenceError	Sai thứ tự
31	requestOutOfRange	Tham số ngoài phạm vi
33	securityAccessDenied	Chưa mở khóa
35	invalidKey	Key sai
36	exceedNumberOfAttempts	Sai quá số lần
37	requiredTimeDelayNotExpired	Chưa hết thời gian chờ
70	uploadDownloadNotAccepted	Chưa xóa vùng đích
71	transferDataSuspended	Vượt kích thước
72	generalProgrammingFailure	Lỗi xóa/ghi/verify
73	wrongBlockSequenceCounter	BSC sai
78	requestCorrectlyReceived-ResponsePending	Cần thêm thời gian
7E	subFunctionNotSupportedInActiveSession	Sub-function không cho phép trong session
7F	serviceNotSupportedInActiveSession	Service không cho phép trong session

21.2 Phụ lục B – Ngoài phạm vi và hướng nâng cấp

Mục	Là gì	Vì sao chưa làm	Nói gì khi được hỏi
AUTOSAR thật	Chuẩn kiến trúc ECU với stack thương mại	Cần tool có license, nhiều tháng	Kiến trúc mượn cách chia module của AUTOSAR
Chữ ký số / mã hóa	Chống firmware giả mạo / chống đọc trộm	Cần thư viện mật mã và quản lý khóa	CRC chống lỗi truyền, không chống giả mạo; nâng cấp bằng ECDSA trong 31 0202
Cập nhật bootloader	Thay chính code ở sector 0-1	Rủi ro brick	Dùng boot updater chạy như app; bình thường khóa ghi vùng boot
Staging / A/B	Giữ app cũ đến khi app mới hợp lệ	Tốn gấp đôi flash, phức tạp	Đã chừa sector 8-11; single slot là cách FBL ô tô phổ biến
Flash driver trong RAM	Driver ghi flash chạy từ RAM, có thể tải qua UDS	Không cần cho F407 ở quy mô này	Tránh CPU dùng khi xóa; là nền của kiến trúc PBL/SBL